```
    Asymptotic one-sample Kolmogorov-Smirnov test

data:  x
D = 0.081988, p-value = 0.5122
alternative hypothesis: two-sided


    Shapiro-Wilk normality test

data:  x
W = 0.97129, p-value = 0.02779
```

- What is the issue with violating the normality assumption?
    - If the violation is due to some large (positive or negative) residuals i.e. outliers, then we should revisit our fitted model. Perhaps we should consider other robust regression (https://en.wikipedia.org/wiki/Robust_regression) techniques that are less sensitive to outliers.
    - Improper calibration of confidence interval, hypothesis testing, and p-values.

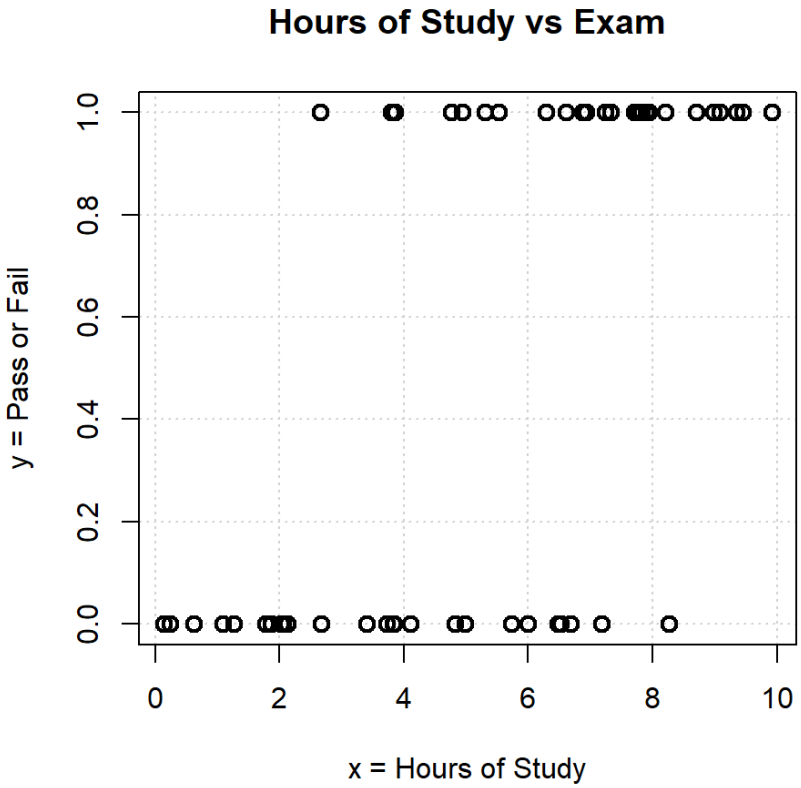
6 Logistic Regression

- **Logistic regression** (https://en.wikipedia.org/wiki/Logistic_regression) is one of the simplest linear models for binary classification. The name “regression” is misleading since it is actually for classification task (with $y \in \{0, 1\}$) and not regression task (for $y \in \mathbb{R}$).
- In statistics and machine learning literature, the logistic regression model can be formulated and viewed in multiple ways:
    1. (Statistics) it is one kind of generalized linear model GLM (https://en.wikipedia.org/wiki/Generalized_linear_model) involving the Bernoulli or binomial distribution within the exponential family (https://en.wikipedia.org/wiki/Exponential_family).
    2. (Statistics) It can be viewed as a latent variable model that thresholds a real-valued random variable as a binary response.
    3. (Machine learning) It can be viewed as a linear combination of input features with sigmoid (https://en.wikipedia.org/wiki/Sigmoid_function) transformation and log loss.
    4. (Machine learning) It can be implemented as a 1-layer fully-connected feed-forward neural net (https://en.wikipedia.org/wiki/Neural_network_(machine_learning)) with softmax (https://en.wikipedia.org/wiki/Softmax_function) activation function and cross-entropy (https://en.wikipedia.org/wiki/Cross-entropy) loss objective function.

6.1 Binary Classification

- For a machine learning predictive task with explanatory variable $\vec{x} \in \mathbb{R}^d$ and response variable y, if the response variable $y \in \{c_1, \ldots, c_k\}$ is categorical / discrete, then it is a **classification** task.
- Moreover, if there are only $k = 2$ classes so $y \in \{c_1, c_2\}$ is binary / **dichotomous** i.e. it takes one of only two possible distinct values / **class labels**, then it is a **binary** classification task.
- *Example:* Let $x \in [0, \infty)$ be the number of hours of studying and $y \in \{\text{pass}, \text{fail}\}$ be the outcome of an exam.

```
study <- c(0.13, 0.23, 0.62, 1.08, 1.26, 1.77, 1.86, 2.02, 2.06, 2.12,
           2.66, 2.67, 3.40, 3.72, 3.80, 3.82, 3.84, 3.86, 4.11, 4.77,
           4.82, 4.94, 4.98, 5.30, 5.53, 5.73, 6.00, 6.29, 6.47, 6.52,
           6.61, 6.68, 6.87, 6.93, 7.18, 7.24, 7.32, 7.70, 7.77, 7.83,
           7.89, 7.94, 8.21, 8.27, 8.70, 8.98, 9.08, 9.35, 9.45, 9.92)
exam <- c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          1, 0, 0, 0, 1, 0, 0, 1, 0, 1,
          0, 1, 0, 1, 1, 0, 0, 1, 0, 0,
          1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
          1, 1, 1, 0, 1, 1, 1, 1, 1, 1) # 1 is pass and 0 is fail
```



- *Example:* Let $\vec{x} \in \mathbb{R}^d$ be the word frequencies of d pre-defined words of interest in an email and $y \in \{\text{legitimate}, \text{spam}\}$.

6.2 Statistical Assumption, Link Function

- Let $(\vec{x}_1, y_1), \ldots, (\vec{x}_n, y_n)$ be n pairs of observed data points with d-dimensional explanatory variable $\vec{x}_i = (x_{i1}, \ldots, x_{id}) \in \mathbb{R}^d$ and binary response variable $y_i \in \{0, 1\}$.
- The statistical model assumption of logistic regression is:
    1. **Deterministic explanatory variable:** Explanatory variables $\vec{x}_i$ is fixed / deterministic / not a random variable in this statistical model.
    2. **Random response variable:** Response variable Y_i is viewed as a random variable. Each observed response value y_i is a realization of its random variable Y_i.
    3. **Independence:** The response variables Y_i's are independent random variables. (In this case, the likelihood function factors into a product of n terms for easier mathematical computation later.)
    4. **Bernoulli distributed:** Since each random variable $Y_i \in \{0, 1\}$ is binary, it must have a Bernoulli distribution

$$Y_i \sim \text{Bern}(p_i)$$

For each ith observed data point, the Bernoulli parameter $p_i \in (0,1)$ can be different.

5. **Linear:** There is an increasing relationship between the probability of occurrence $p_i = P(Y_i = 1)$ (which is same as the expected value $\mu_i = \mathbb{E}(Y_i)$) and linear combination of explanatory variable $\vec{x}_i$.

$$\underbrace{\mathbb{E}(Y_i)}_{\mu_i} = \underbrace{P(Y_i = 1)}_{p_i} \quad \text{vs} \quad \underbrace{\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}}_{\text{linear combination}}$$

If the right-hand side increases, then the left-hand side should increase too, and vice versa. Here the $\beta_0, \ldots, \beta_d$ parameters are common across all n data points. They are unknown and are to be estimated from the data.

6. **Link function:** There is some monotonic increasing function $f : \mathbb{R} \mapsto (0,1)$ that links the right-hand side and left-hand side together.

$$\underbrace{\mathbb{E}(Y_i)}_{\mu_i} = \underbrace{P(Y_i = 1)}_{p_i} = f(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id})$$

For each ith observed data point, since $\vec{x}_i$ can be different, so could p_i.

- Previously for linear regression, since both the expected value $\mathbb{E}(Y_i) \in \mathbb{R}$ and the linear combination $\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id} \in \mathbb{R}$ are real-valued already, there is no need to introduce any link function f between them. (Technically, the link function $f : \mathbb{R} \mapsto \mathbb{R}$ is just the identity function i.e. $f(x) = x$.)
- Now for logistic regression, we need some monotonic increasing function $f : \mathbb{R} \mapsto (0,1)$ to "squeeze" the entire real number line $\mathbb{R}$ onto the $(0,1)$ interval. Such a function must be non-linear. There is actually a natural / default / **canonical** choice for this function f.
- Recall $p_i = P(Y_i = 1)$ is the probability of occurrence of an event. The **odds** of this event is

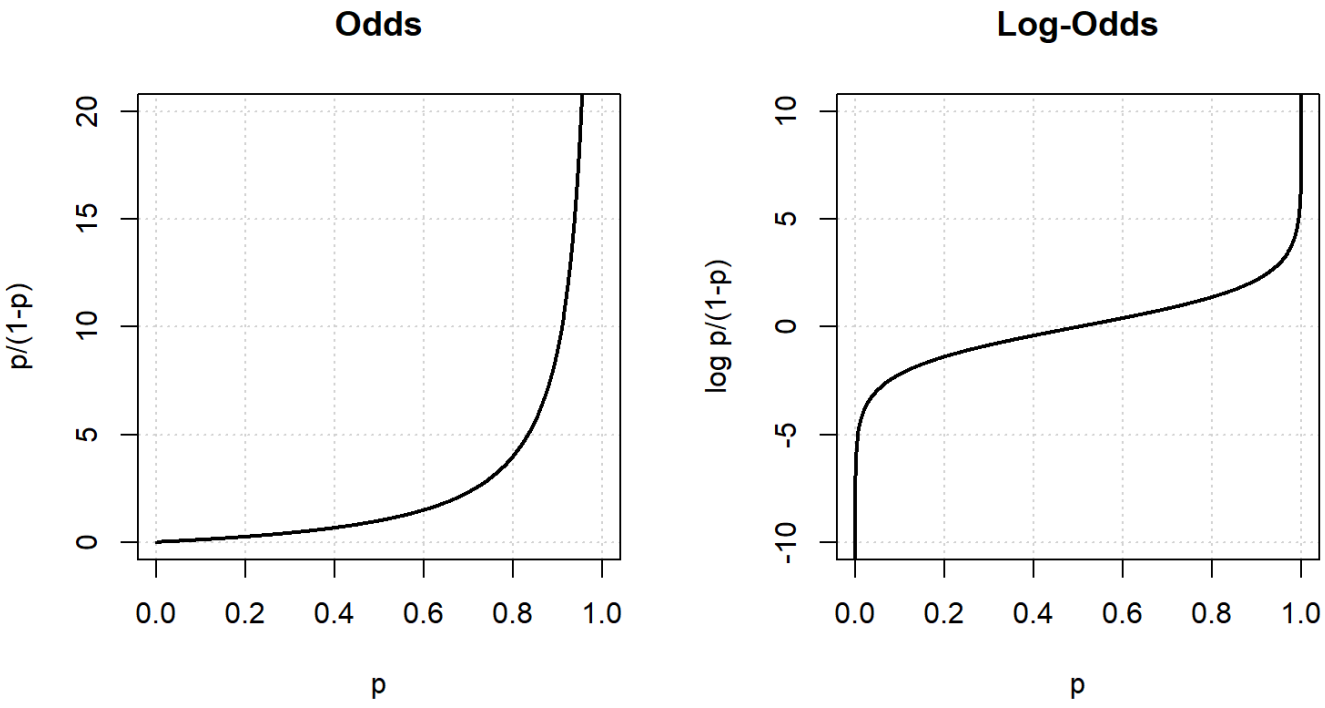
$$\frac{p_i}{1 - p_i} \in (0, \infty)$$

It is a non-negative number. If $p_i \to 0$, then the odds goes down to 0. If $p_i = 1/2$, then the odds is 1. If $p_i \to 1$, then the odds goes up to ∞.

- *Example:* In gambling, if the probability of winning is $1/3$, then the odds is $(1/3)/(1 - 1/3) = 1/2$ which is $1:2$ i.e. "one-to-two".
- The **log-odds** is the (natural base e not base 10) logarithm of the odds i.e.

$$\log \frac{p_i}{1 - p_i} \in (-\infty, \infty) = \mathbb{R}$$

It is a real-valued number. If $p_i \to 0$, then the log-odds goes down to $-\infty$. If $p_i = 1/2$, then the log-odds is 0. If $p_i \to 1$, then the log-odds goes up to ∞.



- Main idea is to use the real-valued linear combination of the explanatory variable $\overrightarrow{x_i}$ to predict the real-valued log-odds i.e.

$$\log \frac{p_i}{1-p_i} = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}$$

- The transformation from probability p_i to its log-odds is called the **logit** (https://en.wikipedia.org/wiki/Logit) (as an abbreviation of “logistic unit”) function

$$\operatorname{logit}(p_i) = \log \frac{p_i}{1-p_i}$$

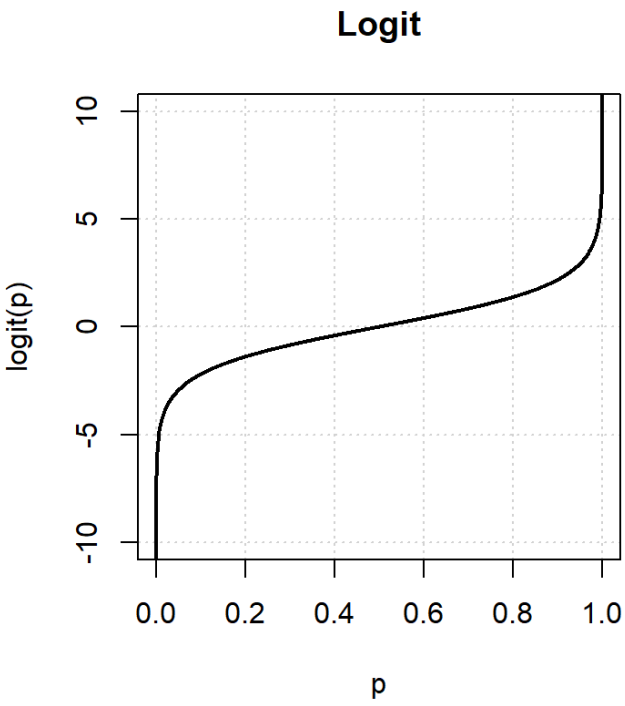
  The logit function is invertible. Its inverse is the **expit** function

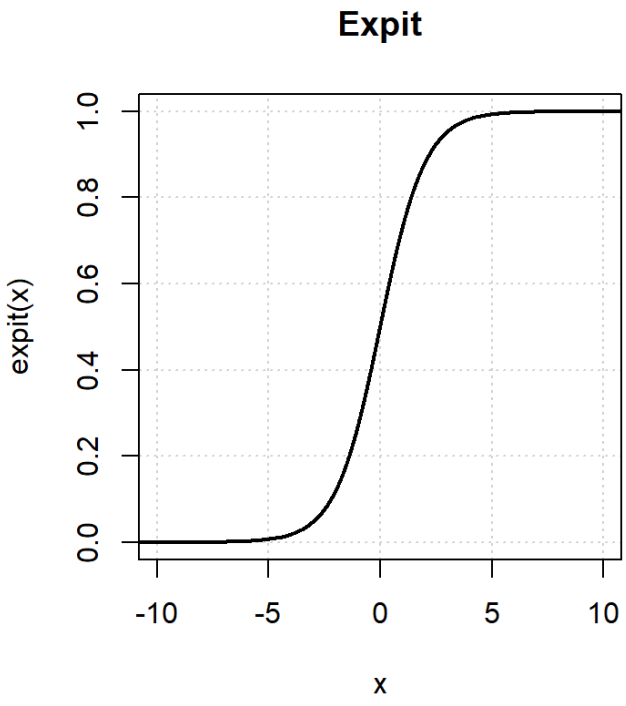
$$p_i = \operatorname{expit}(x) = \frac{e^x}{1+e^x} = \frac{1}{1+e^{-x}}$$

- Expit function is monotonic increasing and squeezes the entire real number line $\mathbb{R}$ onto the $(0,1)$ interval. In general, such an S-shape curve is called a **sigmoid function** (https://en.wikipedia.org/wiki/Sigmoid_function). (In machine learning, some people even refer to this expit function as “the” sigmoid function.)
- Fact: The derivative of the expit function is

$$\frac{dp}{dx} = \frac{d}{dx}\operatorname{expit}(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1}{1+e^{-x}}\right)\left(\frac{e^{-x}}{1+e^{-x}}\right) = p(1-p)$$

- Expit (and also logit) function is symmetric in the sense that as $x \to -\infty$ or $x \to +\infty$, both $f(x) \to 0$ and $f(x) \to 1$ at the same rate.





- Continuing the statistical model assumption:
  7. **Logit as link:** Combining the above together, the logistic regression model equation is

$$\log \frac{p_i}{1-p_i} = \text{logit}(p_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}$$

Equivalently, it is

$$\underbrace{\mathbb{E}(Y_i)}_{\mu_i} = \underbrace{P(Y_i = 1)}_{p_i} = \underbrace{\text{expit}}_{f}(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id})}}$$

(Technically, in statistics literature, our function $f = \text{expit}$ is not referred to as the link function. Instead, its inverse function $f^{-1} = \text{logit}$ is the link function.)

- In general, any S-shape sigmoid function $f : \mathbb{R} \mapsto (0, 1)$ that squeezes the entire real number line $\mathbb{R}$ onto the $(0, 1)$ interval can be used as a link function for logistic regression. In statistics literature, some other notable choices of link function are:
  1. **Probit** (https://en.wikipedia.org/wiki/Probit)

$$f(x) = \Phi(x) = \int_{\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}\, dz$$

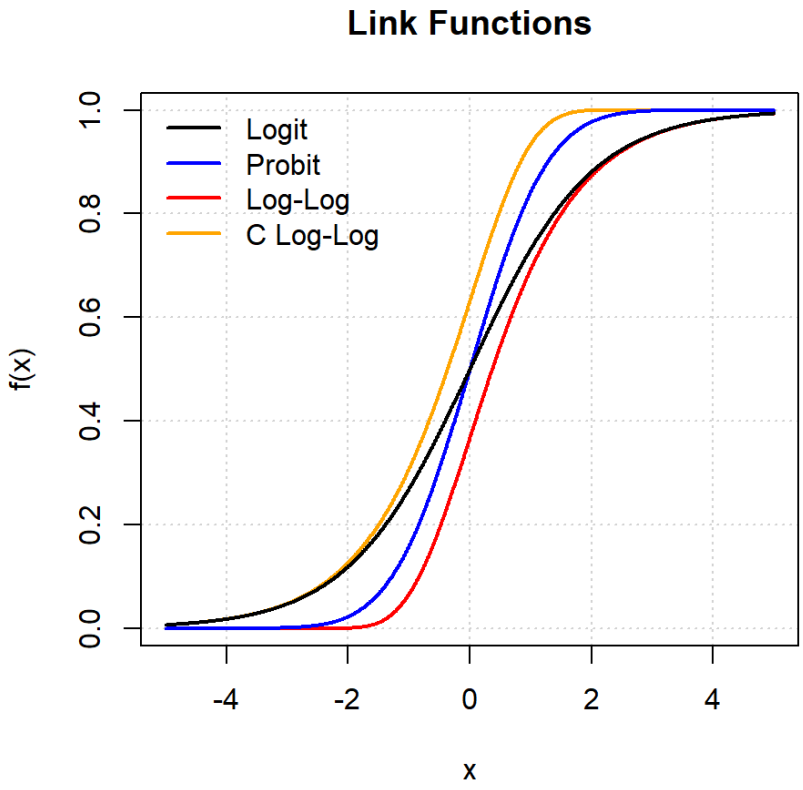
This is just the cumulative distribution function CDF of the standard normal $\mathcal{N}(0, 1)$ distribution. Using this link is called the **probit regression** (https://en.wikipedia.org/wiki/Probit_model). It has an interpretation of some latent variable model that thresholds a normally distributed value as a binary response. (Don't worry about it.) The profit link function $\Phi(x)$ is symmetric (like the logit link function previously). Since $\Phi(x)$ does not have a closed-form formula, it is slightly more expensive to compute than the logit link function.

  2. **Log-log** and **complementary log-log**

$$f(x) = e^{-e^{-x}} \quad \text{and} \quad f(x) = 1 - e^{-e^{x}}$$

These are the cumulative distribution function CDF of the log-Weibull (https://en.wikipedia.org/wiki/Weibull_distribution) extreme value distribution (https://en.wikipedia.org/wiki/Generalized_extreme_value_distribution) and its negative sibling. The log-log and

complementary log-log link functions are asymmetric. For the complementary log-log link function, as $x \to -\infty$ or $x \to +\infty$, $f(x) \to 0$ at a slower rate than $f(x) \to 1$. On the other hand, for the log-log link function, $f(x) \to 0$ at a faster rate than $f(x) \to 1$. Therefore, these link functions are useful for modeling rare events whose probabilities of occurrence tend to be very small (or very large) e.g. survival analysis, hazard ratio.

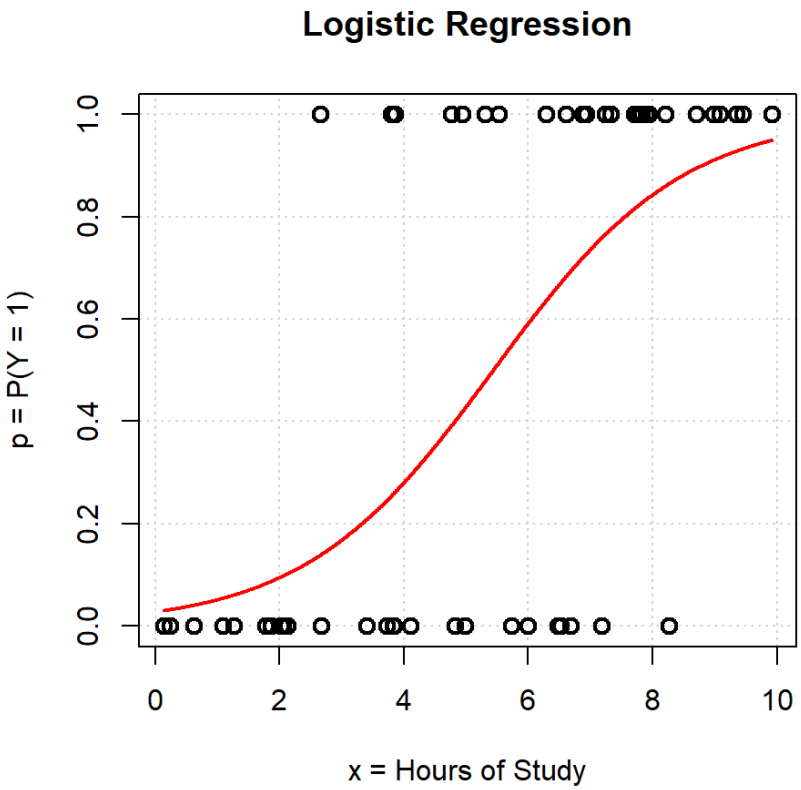


6.3 Likelihood Inference for $\vec{\beta}$, Log Loss

- Logistic regression with logit link function is the statistical model

$$Y_i \sim \text{Bern}\Big(\underbrace{\frac{1}{1+e^{-(\beta_0+\beta_1 x_{i1}+\cdots+\beta_d x_{id})}}}_{0<p_i<1}\Big) \quad \text{for } i = 1, \ldots, n$$

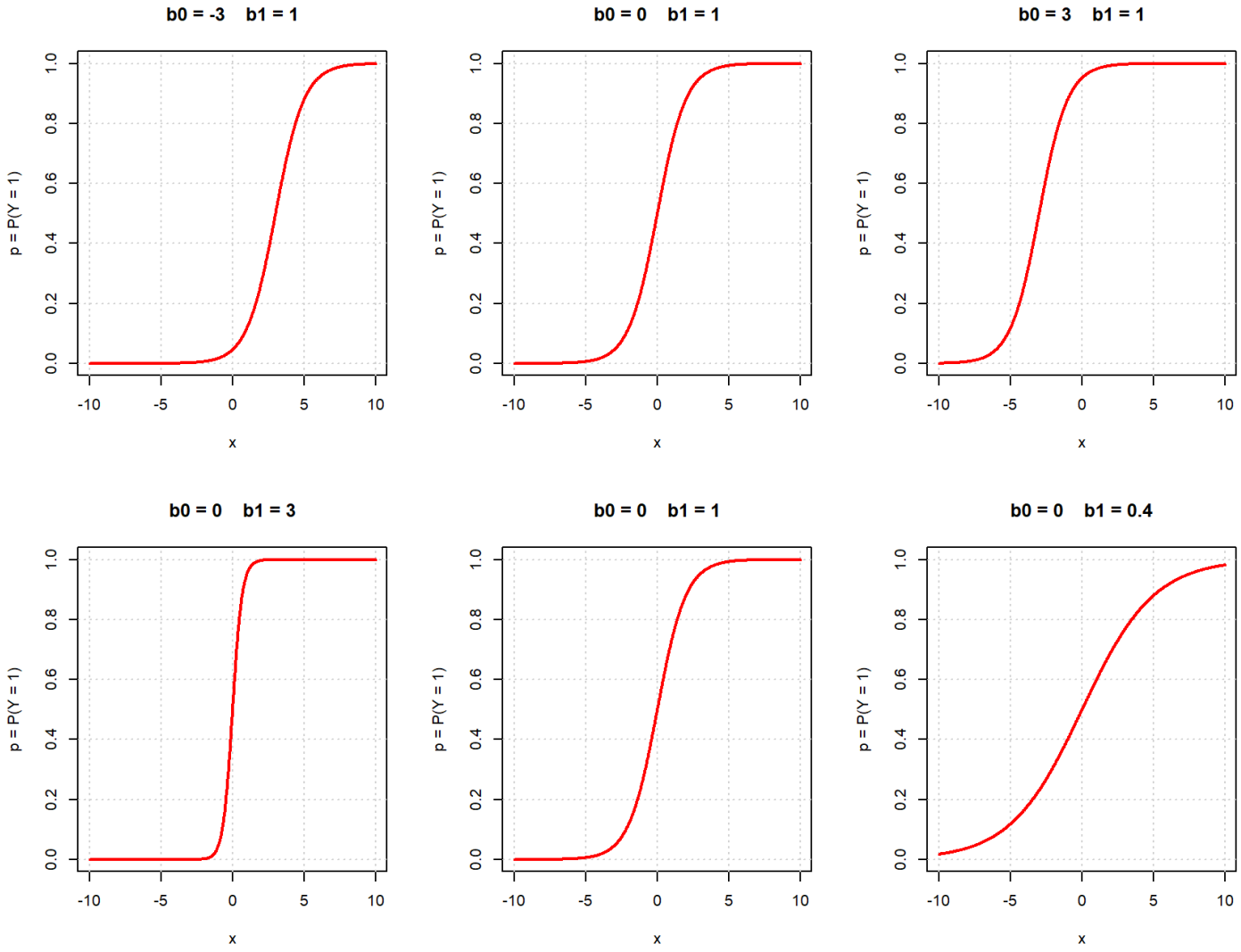
  with parameters $\beta_0, \ldots, \beta_d$.
- Given n pairs of observed data points $(\vec{x}_1, y_1), \ldots, (\vec{x}_n, y_n)$, we wish to "fit the best line / curve to the data" by choosing the optimal $\beta_0^*, \ldots, \beta_d^*$ parameters. Since the model involves a non-linear link function, this "line" is not a straight line but a sigmoid curve.



- How do the choices of $\beta_0, \ldots, \beta_d$ parameters affect the resulting sigmoid curve? Let's consider a simple case with only one explanatory variable x i.e. $d = 1$ so there are only the "intercept" β_0 and "slope" β_1 parameters. In this case, the model equation is

$$P(Y_i = 1) = p_i = \text{expit}(\beta_0 + \beta_1 x_i) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_i)}} \quad \text{for } i = 1, \ldots, n$$

  Below are some illustrations with various choices of β_0 and β_1 parameters. In summary, β_0 controls the horizontal shift of the sigmoid curve and β_1 controls the steepness of the S-shape. (They are analogous to the concepts of intercept and slope in the case of a straight line.)



- To find the optimal $\beta_0^*, \ldots, \beta_d^*$ parameters such that the fitted curve is "closest" to all observed data points, we will NOT use the mean squared error MSE to compare the observed binary response $y_i \in \{0, 1\}$ vs the predicted probability of occurrence $p_i \in (0, 1)$:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - p_i)^2$$

Why not?

1. **Non-convex optimization:** For linear regression model, the MSE objective / loss / cost function is a quadratic function of the $\beta_0, \ldots, \beta_d$ parameters and hence a convex optimization (https://en.wikipedia.org/wiki/Convex_optimization) problem which is easy to optimize with unique global optimum $\beta_0^*, \ldots, \beta_d^*$. On the other hand, for logistic regression with the logit link function, if we use MSE as the objective function, then the corresponding optimization problem is not convex so it can be difficult to solve numerically.
2. **Non-linear perception of probability**: We tend to think of $p_i = 0.0001$ vs $p_i = 0.01$ as a $100\times$ increase in the probability of occurrence of a rare event. On the other hand, $p_i = 0.5001$ vs $p_i = 0.51$ seem negligible.
3. **Insufficient penalty for misclassification:** If the observed response $y_i = 1$ but the predicted probability $p_i \approx 0$, then we misclassify badly but only incur a penalty of $(y_i - p_i)^2 \approx 1$ which is often not severe enough.

- Instead, we follow the maximum likelihood principle to estimate the $\beta_0, \ldots, \beta_d$ parameters. Recall the likelihood function for Bernoulli distribution is

$$L(\beta_0, \ldots, \beta_d) = \prod_{i=1}^{n} P(Y_i = y_i) = \prod_{i=1}^{n} p_i^{y_i}(1 - p_i)^{1-y_i} \quad \text{with} \quad p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id})}}$$

Therefore, $L(\beta_0, \ldots, \beta_d)$ is indeed a function of the $\beta_0, \ldots, \beta_d$ parameters. Now the log-likelihood function is

$$l(\beta_0, \ldots, \beta_d) = \log L(\beta_0, \ldots, \beta_d) = \sum_{i=1}^{n} y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

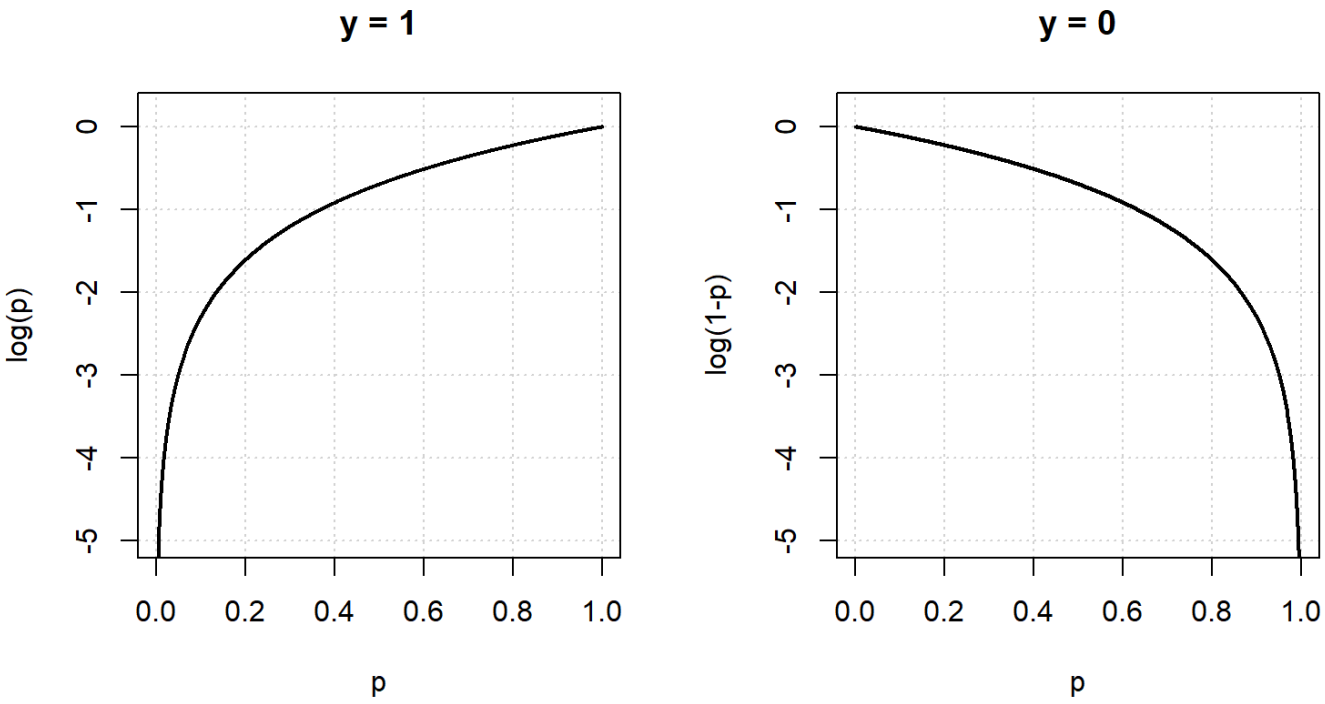
The maximum likelihood estimate MLE is

$$\beta_0^*, \ldots, \beta_d^* = \underset{\beta_0, \ldots, \beta_d}{\arg\max}\, l(\beta_0, \ldots, \beta_d) = \underset{\beta_0, \ldots, \beta_d}{\arg\max} \sum_{i=1}^{n} y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

- Why does maximizing the likelihood function make sense? The contribution of the ith data point $(\vec{x}_i, y_i)$ to the log-likelihood function $l(\beta_0, \ldots, \beta_d)$ is

$$y_i \log(p_i) + (1 - y_i) \log(1 - p_i) = \begin{cases} \log(p_i) & \text{if } y_i = 1 \\ \log(1 - p_i) & \text{if } y_i = 0 \end{cases}$$

If $y_i = 1$, then the contribution is maximized when $p_i = 1$. Similarly for $y_i = 0$ and $p_i = 0$. So it encourages choosing $\beta_0, \ldots, \beta_d$ parameters to bring $y_i \approx p_i$ close to each other. Moreover, the penalty of an overly-confident prediction mistake can be severe e.g. if $y_i = 1$ but $p_i \approx 0$, then $\log(p_i) \approx -\infty$.



- In machine learning literature, the **log loss** / **cross-entropy loss** is

$$\text{loss}(y, p) = -\left(y \log(p) + (1 - y) \log(1 - p)\right)$$

It is the most common objective / loss / cost function for binary classification task. This is exactly the negative version of the log-likelihood function above i.e. maximizing for the MLE is same as minimizing the log loss.

- Although the MLE optimization problem

$$\beta_0^*, \ldots, \beta_d^* = \underset{\beta_0, \ldots, \beta_d}{\arg\max} \sum_{i=1}^{n} y_i \log(p_i) + (1 - y_i) \log(1 - p_i)$$

is convex and thus must have a unique global optimum, there is unfortunately no closed-form solution (unlike the linear regression which has known explicit matrix formula for $\vec{\beta}_{\text{MLE}}$.)

- In practice, we solve this optimization using some iterative algorithm. For example, R's `glm()` function uses the iteratively reweighted least squares IRLS (https://en.wikipedia.org/wiki/Iteratively_reweighted_least_squares) / Fisher (https://en.wikipedia.org/wiki/Fisher_information) scoring (https://en.wikipedia.org/wiki/Scoring_algorithm) algorithm which is essentially the Newton-Raphson method (https://en.wikipedia.org/wiki/Newton%27s_method). Other iterative approaches include gradient descent (https://en.wikipedia.org/wiki/Gradient_descent) (which is popular within the machine learning community) and L-BFGS (https://en.wikipedia.org/wiki/Limited-memory_BFGS).
- In R, use the `glm(..., family=binomial(...))` function to fit a logistic regression model (which is one kind of generalized linear model GLM (https://en.wikipedia.org/wiki/Generalized_linear_model)).

```
study <- c(0.13, 0.23, 0.62, 1.08, 1.26, 1.77, 1.86, 2.02, 2.06, 2.12,
           2.66, 2.67, 3.40, 3.72, 3.80, 3.82, 3.84, 3.86, 4.11, 4.77,
           4.82, 4.94, 4.98, 5.30, 5.53, 5.73, 6.00, 6.29, 6.47, 6.52,
           6.61, 6.68, 6.87, 6.93, 7.18, 7.24, 7.32, 7.70, 7.77, 7.83,
           7.89, 7.94, 8.21, 8.27, 8.70, 8.98, 9.08, 9.35, 9.45, 9.92)
exam <- c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          1, 0, 0, 0, 1, 0, 0, 1, 0, 1,
          0, 1, 0, 1, 1, 0, 0, 1, 0, 0,
          1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
          1, 1, 1, 0, 1, 1, 1, 1, 1, 1) # 1 is pass and 0 is fail

model <- glm(exam ~ study, family = binomial) # Logit link is default. See ?family

# Link function
model$family$link # See str(model)

# Beta's
model
model$coefficients
coef(model)
coefficients(model)
summary(model)
```

```
[1] "logit"
```

```


Call:  glm(formula = exam ~ study, family = binomial)

Coefficients:
(Intercept)        study
    -3.5686       0.6563

Degrees of Freedom: 49 Total (i.e. Null);  48 Residual
Null Deviance:      69.31
Residual Deviance: 46.72    AIC: 50.72
```

```
(Intercept)       study
  -3.568573    0.656332
```

```
(Intercept)       study
  -3.568573    0.656332
```

```
(Intercept)       study
  -3.568573    0.656332
```

```
Call:
glm(formula = exam ~ study, family = binomial)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -3.5686     1.0652   -3.35 0.000808 ***
study         0.6563     0.1793    3.66 0.000252 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 69.315  on 49  degrees of freedom
Residual deviance: 46.716  on 48  degrees of freedom
AIC: 50.716

Number of Fisher Scoring iterations: 5
```

- In `glm(..., family=binomial)`, if the response variable is a logical vector of `TRUE` / `FALSE` values, then it would also be viewed as 1's and 0's. If the response variable is an R's `factor`, then its first level is viewed as 0 and all other levels are viewed as 1. For example, `'success'` is 1 and `'failure'` is 0 based on alphabetical order.

```
# Same data as previously
study <- c(0.13, 0.23, 0.62, 1.08, 1.26, 1.77, 1.86, 2.02, 2.06, 2.12,
           2.66, 2.67, 3.40, 3.72, 3.80, 3.82, 3.84, 3.86, 4.11, 4.77,
           4.82, 4.94, 4.98, 5.30, 5.53, 5.73, 6.00, 6.29, 6.47, 6.52,
           6.61, 6.68, 6.87, 6.93, 7.18, 7.24, 7.32, 7.70, 7.77, 7.83,
           7.89, 7.94, 8.21, 8.27, 8.70, 8.98, 9.08, 9.35, 9.45, 9.92)
exam <- c('f', 'f', 'f', 'f', 'f', 'f', 'f', 'f', 'f', 'f',
          'p', 'f', 'f', 'f', 'p', 'f', 'f', 'p', 'f', 'p',
          'f', 'p', 'f', 'p', 'p', 'f', 'f', 'p', 'f', 'f',
          'p', 'f', 'p', 'p', 'f', 'p', 'p', 'p', 'p', 'p',
          'p', 'p', 'p', 'f', 'p', 'p', 'p', 'p', 'p', 'p') # 'p' is pass and 'f' is fail

# Verify the default (alphabetical) ordering of levels
levels(factor(exam))
```

```
[1] "f" "p"
```

```
glm(factor(exam) ~ study, family = binomial) # Same model as previously
```

```
Call:  glm(formula = factor(exam) ~ study, family = binomial)

Coefficients:
(Intercept)        study
    -3.5686       0.6563

Degrees of Freedom: 49 Total (i.e. Null);  48 Residual
Null Deviance:      69.31
Residual Deviance: 46.72    AIC: 50.72
```

- There are two other ways to format the response variable in `glm(..., family=binomial)` for grouped data (later).

- Interpretation of coefficient β_j: If x_j increases by $+1$ unit to become $x_j + 1$ while all other explanatory variables do not change, then the odds become

$$\underbrace{\frac{p_i^{\text{new}}}{1-p_i^{\text{new}}}}_{\text{new odds}} = e^{\beta_0+\beta_1 x_{i1}+\cdots+\beta_j(x_{ij}+1)+\cdots\beta_d x_{id}} = \underbrace{\frac{p_i^{\text{old}}}{1-p_i^{\text{old}}}}_{\text{old odds}} \times e^{\beta_j}$$

- This does not mean the probability p_i itself changes by a constant amount of e^{β_j}. The amount of change in p_i depends on the specific initial $\vec{x}_i$ value.

```
b0 <- 0
b1 <- 1

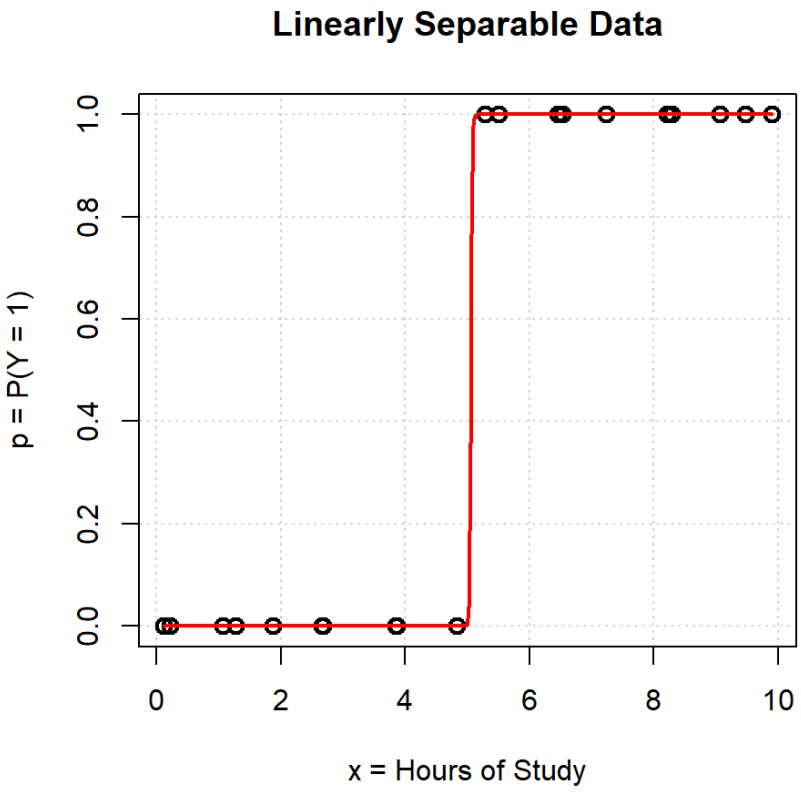
x <- c(0, 1) # From 0 to 1
(p <- 1 / (1 + exp(-(b0 + b1 * x))))

x <- c(3, 4) # From 3 to 4
(p <- 1 / (1 + exp(-(b0 + b1 * x))))
```

```
[1] 0.5000000 0.7310586
[1] 0.9525741 0.9820138
```

- Many topics from linear regression continue to apply to logistic regression e.g. invariance of the model to affine transformation of an explanatory variable $x_j \mapsto a + bx_j$, one-hot encoding of categorical variable, interaction term, variable transformation / feature engineering e.g. polynomial, multicollinearity, and variance inflation factor VIF.
- Fisher scoring algorithm usually converges very quickly to the optimal $\vec{\beta}_{\text{MLE}}$ after only a small number of iterations.
- However, ironically if the data points are **linearly separable** (https://en.wikipedia.org/wiki/Linear_separability), then Fisher scoring algorithm would never converge even after a large number of (R's default is 25) iterations. In this case, besides R's warning message, the `Std. Error` of the estimated coefficients shown in the model's `summary()` tend to be very large. Numerically, the reason for non-convergence is because the "slope" parameter $\beta_1 \to \infty$ or $-\infty$ increases without bound in order to make the slope of the sigmoid curve even steeper (or more vertical) so it could fit the data points even better.

```
study <- c(0.11, 0.22, 1.07, 1.27, 1.87, 2.67, 2.68, 3.85, 3.87, 4.83,
           5.28, 5.51, 6.46, 6.53, 7.23, 8.22, 8.29, 9.06, 9.47, 9.90)
midterm <- c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
             1, 1, 1, 1, 1, 1, 1, 1, 1, 1) # 1 is pass and 0 is fail
```




```
model <- glm(midterm ~ study, family = binomial)

summary(model)
model$iter # Fisher scoring number of iteration
model$converged # Fisher scoring converged or not?
```

```
Warning: glm.fit: algorithm did not converge
```

```
Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
Call:
glm(formula = midterm ~ study, family = binomial)

Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)   -458.30  259786.19  -0.002    0.999
study           90.65   51230.16   0.002    0.999

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 2.7726e+01  on 19  degrees of freedom
Residual deviance: 5.5508e-09  on 18  degrees of freedom
AIC: 4

Number of Fisher Scoring iterations: 25

[1] 25
[1] FALSE
```

6.4 Prediction

- R has various functionalities to compute both fitted / predicted probability value $\hat{p}_i$ and its corresponding linear combination $\beta_0^* + \beta_1^* x_1 + \cdots + \beta_d^* x_d$ before the link function.

```
study <- c(0.13, 0.23, 0.62, 1.08, 1.26, 1.77, 1.86, 2.02, 2.06, 2.12,
           2.66, 2.67, 3.40, 3.72, 3.80, 3.82, 3.84, 3.86, 4.11, 4.77,
           4.82, 4.94, 4.98, 5.30, 5.53, 5.73, 6.00, 6.29, 6.47, 6.52,
           6.61, 6.68, 6.87, 6.93, 7.18, 7.24, 7.32, 7.70, 7.77, 7.83,
           7.89, 7.94, 8.21, 8.27, 8.70, 8.98, 9.08, 9.35, 9.45, 9.92)
exam <- c(0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          1, 0, 0, 0, 1, 0, 0, 1, 0, 1,
          0, 1, 0, 1, 1, 0, 0, 1, 0, 0,
          1, 0, 1, 1, 0, 1, 1, 1, 1, 1,
          1, 1, 1, 0, 1, 1, 1, 1, 1, 1) # 1 is pass and 0 is fail

model <- glm(exam ~ study, family = binomial)

# Fitted values / predictions (for probability p_i)
fitted(model)
fitted.values(model)
model$fitted.values
```

```
         1          2          3          4          5          6
0.02979261 0.03174943 0.04063481 0.05418033 0.06056313 0.08265070
         1          2          3          4          5          6
0.02979261 0.03174943 0.04063481 0.05418033 0.06056313 0.08265070
         1          2          3          4          5          6
0.02979261 0.03174943 0.04063481 0.05418033 0.06056313 0.08265070
```

```
# Fitted linear predictor i.e. b0 + b1*x1 + ... + b_d*x_d before link function
b0 <- coef(model)[1]
b1 <- coef(model)[2]
b0 + b1 * study

model$linear.predictors
```

```
[1] -3.483249 -3.417616 -3.161647 -2.859734 -2.741594 -2.406865
        1         2         3         4         5         6
-3.483249 -3.417616 -3.161647 -2.859734 -2.741594 -2.406865
```

```
# Compute fitted probability p_i after logit link
1 / (1 + exp(-model$linear.predictors))
```

```
         1          2          3          4          5          6
0.02979261 0.03174943 0.04063481 0.05418033 0.06056313 0.08265070
```

- Similarly, given a new explanatory variable value, R's `predict()` function can output either the `'response'` probability p_i or its corresponding linear combination before `'link'` function.

```
x.new <- 3

b0 + b1 * x.new
(u <- predict(model, newdata=data.frame(study=x.new), type='link')) # b0 + b1*x1 + ... + b_d*x_d
```

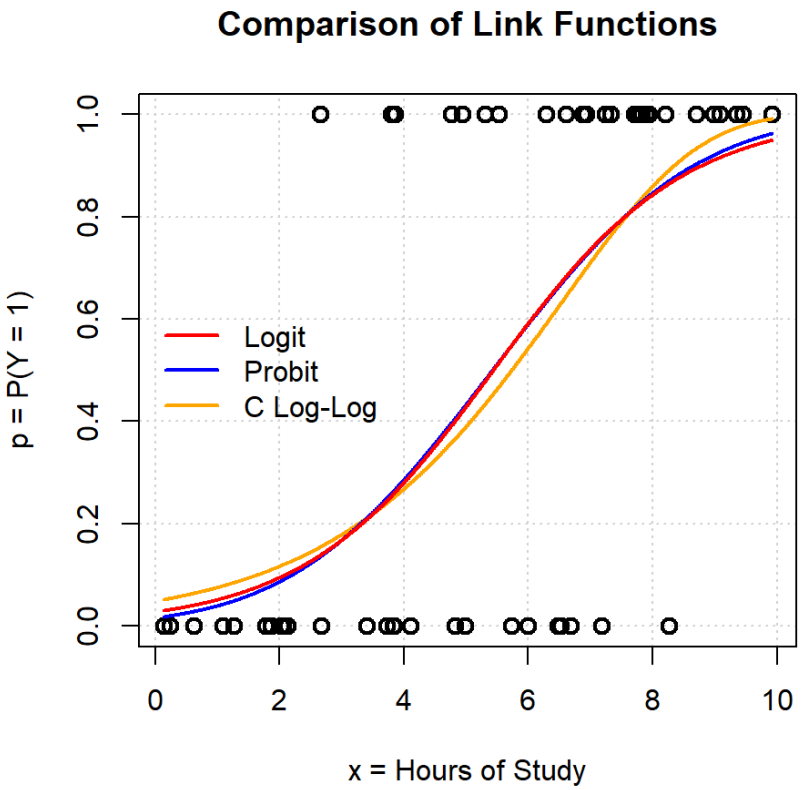
```
(Intercept)
  -1.599577
        1
-1.599577
```

```
1 / (1 + exp(-u)) # Compute logit link
predict(model, newdata=data.frame(study=x.new), type='response') # p_i
```

```
        1
0.1680408
        1
0.1680408
```

- Recall both logit and probit link functions are symmetric around their two tails. Although the probit link $\Phi(x) \to 0$ or 1 as $x \to \pm\infty$ at a faster rate than the logit link, in practice they yield very similar models and predictions.
- On the other hand, use asymmetric log-log or complementary log-log links only if the data exhibit such asymmetry.

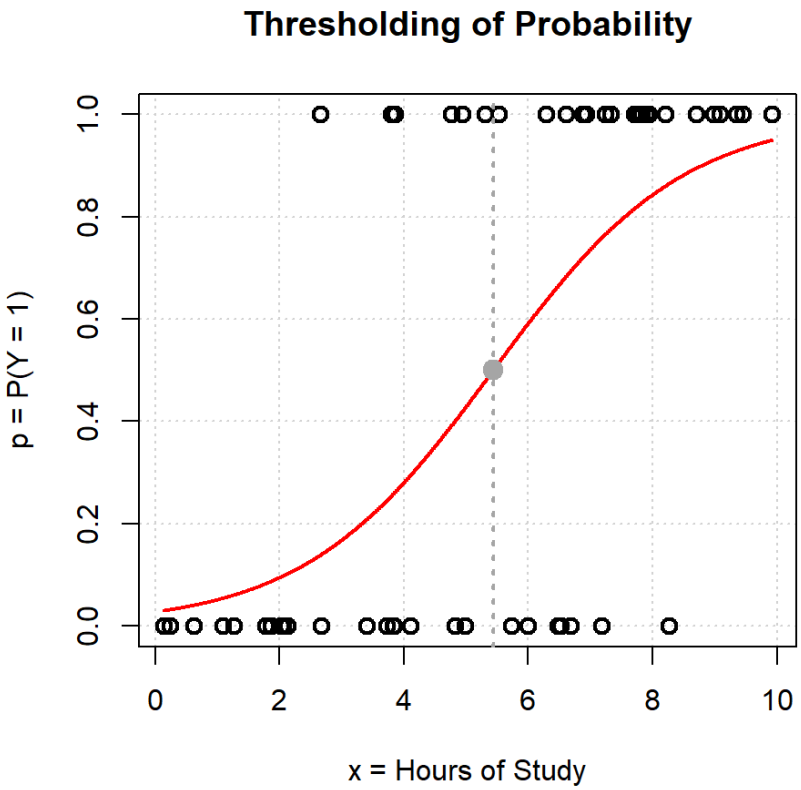
```
logit.model <- glm(exam ~ study, family = binomial(link='logit'))
probit.model <- glm(exam ~ study, family = binomial(link='probit'))
cloglog.model <- glm(exam ~ study, family = binomial(link='cloglog'))
```



6.5 Confusion Matrix

- Logistic regression model only outputs / predicts the probability $\hat{p}_i = P(Y_i = 1)$ of occurrence and not an actual binary class label $\hat{y}_i \in \{0, 1\}$. To do so, we need to choose a threshold $0 < \tau < 1$ to truncate the probability value i.e.

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{p}_i > \tau \\ 0 & \text{if } \hat{p}_i \leq \tau \end{cases}$$



- A **confusion matrix** (https://en.wikipedia.org/wiki/Confusion_matrix) is a 2×2 contingency table (https://en.wikipedia.org/wiki/Contingency_table) to tabulate the observed responses y_i vs the predicted labels $\hat{y}_i$ in a data set.

```
tau <- 0.5
y.hat <- as.numeric(fitted(model) > tau) # Convert TRUE/FALSE to 0/1

(confusion.matrix <- table(exam, y.hat))
```

```
    y.hat
exam  0  1
   0 18  7
   1  6 19
```

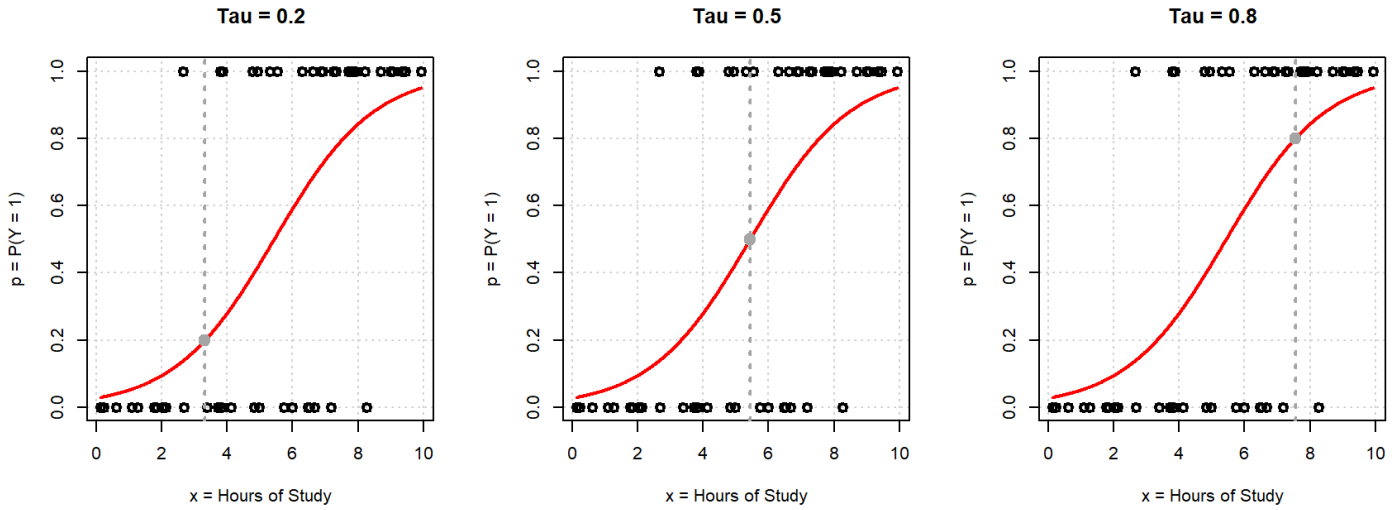
- In machine learning, the most important metrics from a confusion matrix are:
  - Accuracy / error rate / misclassification rate
  - **True positive rate** TPR (https://en.wikipedia.org/wiki/True_positive_rate) and **false positive rate** FPR (https://en.wikipedia.org/wiki/False_positive_rate): For receiver operating characteristic ROC (https://en.wikipedia.org/wiki/Receiver_operating_characteristic) curve and its area under curve AUC (https://en.wikipedia.org/wiki/Receiver_operating_characteristic#Area_under_the_curve) metric.
  - **Precision** (https://en.wikipedia.org/wiki/Precision_and_recall) and **recall** (https://en.wikipedia.org/wiki/Precision_and_recall): For precision-recall curve and F_1-score (https://en.wikipedia.org/wiki/F-score). These are particularly useful for label-imbalanced data set e.g. fraud detection, rare disease.

```
n <- sum(confusion.matrix) # Total number of data points
tn <- confusion.matrix[1,1] # True negative
fp <- confusion.matrix[1,2] # False positive
fn <- confusion.matrix[2,1] # False negative
tp <- confusion.matrix[2,2] # True positive

(accuracy <- (tp + tn) / n) # Accuracy = 1 - error (or misclassification) rate
(tpr <- tp / (tp + fn)) # True positive rate = recall
(fpr <- fp / (fp + tn)) # False positive rate
(precision <- tp / (tp + fp)) # Precision
```

```
[1] 0.74
[1] 0.76
[1] 0.28
[1] 0.7307692
```

- *Example:* For email spam filtering as a binary classificiation task i.e. $y \in \{1 = \text{spam}, 0 = \text{good}\}$:
  - Accuracy: Out of all emails, how many are classified correctly?
  - True positive rate TPR / recall: Out of all truly spam emails, how many are flagged as spam correctly?
  - False positive rata FPR: Out of all truly good emails, how many are flagged as spam incorrectly?
  - Precision: Out of all emails that are flagged as spam, how many are truly spam emails?
- Choice of threshold τ is not necessarily always $1/2$. It should be a business decision (rather than a scientific decision) based on the risk of false positive vs false negative error.
- As τ increases from 0 to 1, it becomes more conservative / reluctant to classify a data point as $\hat{y}_i = 1$ unless its probability $\hat{p}_i$ is sufficiently high. In doing so:
  - More data points are classified as $\hat{y}_i = 0$ i.e. negative.
  - Accuracy could increase or decrease i.e. it could make fewer or more misclassification errors. This is a trade-off between false positive FP vs false negative FN mistakes.
  - Both true positive rate TPR (i.e. recall) and false positive rate FPR must decrease.
  - Precision could increase or decrease. But as $\tau \approx 1$, precision usually ≈ 1.




```
for (tau in c(0.2, 0.5, 0.8)) {
  y.hat <- as.numeric(fitted(model) > tau)
  confusion.matrix <- table(exam, y.hat)
  ...
  c(accuracy, tpr, fpr, precision)
}
```

```
# tau <- 0.2
    y.hat
exam  0  1
   0 11 14
   1  1 24
[1] 0.7000000 0.9600000 0.5600000 0.6315789
```

```
# tau <- 0.5
    y.hat
exam  0  1
   0 18  7
   1  6 19
[1] 0.7400000 0.7600000 0.2800000 0.7307692
```

```
# tau <- 0.8
    y.hat
exam  0  1
   0 24  1
   1 13 12
[1] 0.7200000 0.4800000 0.0400000 0.9230769
```

6.6 Decision Boundary

- Each threshold τ value induces a linear **decision boundary** (https://en.wikipedia.org/wiki/Decision_boundary). To understand this linearity, note a choice of threshold τ on the probability p_i

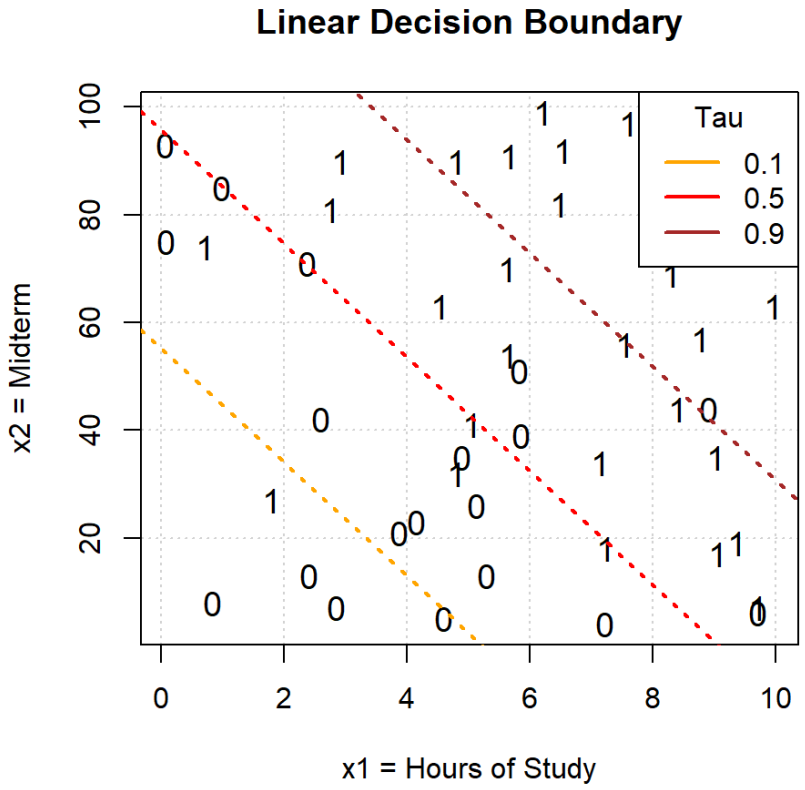
$$\tau < p_i = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id})}}$$

  is equivalent to another related threshold on the linear combination of explanatory variables itself i.e.

$$\underbrace{\log \frac{\tau}{1-\tau}}_{\text{another threshold}} < \beta_0 + \beta_1 x_{i1} + \cdots + \beta_d x_{id}$$

  This last equation represents a (linear) line / plane / (high-dimensional) hyperplane (https://en.wikipedia.org/wiki/Hyperplane) in $\mathbb{R}^d$ where $\vec{x}_i = (x_{i1}, \ldots, x_{id})$ sits.
- It is easier to visualize such linear decision boundary in the case with just two continuous explanatory variables x_1 and x_2 so the "input space" is the $\mathbb{R}^2$ plane.
- *Example:* Let $x_1 \in [0, \infty)$ be the number of hours of studying, $x_2 \in [0, 100]$ be the midterm score, and $y \in \{\text{pass}, \text{fail}\}$ is the outcome of the final exam.

```
data <- data.frame(study=c(5.86, 0.09, 2.94, 2.77, 8.14, 2.60, 7.24, 9.06, 9.49, 0.73,
                           7.55, 2.86, 1.00, 9.54, 4.16, 4.55, 9.71, 5.84, 9.62, 7.62,
                           7.15, 9.97, 5.06, 4.90, 6.49, 8.31, 4.82, 8.42, 5.14, 5.30,
                           5.67, 2.39, 8.78, 6.55, 4.82, 9.71, 4.60, 6.22, 3.88, 0.07,
                           9.38, 2.42, 5.66, 1.81, 9.04, 0.84, 9.01, 8.92, 7.23, 5.65),
                   midterm=c(39, 75, 90, 81, 82, 42, 18, 17, 89, 74,
                             56, 7, 85, 91, 23, 63, 7, 51, 81, 97,
                             34, 63, 41, 35, 82, 69, 32, 44, 26, 13,
                             91, 71, 57, 92, 90, 6, 5, 99, 21, 93,
                             19, 13, 54, 27, 35, 8, 77, 44, 4, 70),
                   exam=c(0, 0, 1, 1, 1, 0, 1, 1, 1, 1,
                          1, 0, 0, 1, 0, 1, 1, 0, 1, 1,
                          1, 1, 1, 0, 1, 1, 1, 1, 0, 0,
                          1, 0, 1, 1, 1, 0, 0, 1, 0, 0,
                          1, 0, 1, 1, 1, 0, 1, 0, 0, 1)) # 1 is pass and 0 is fail
```



```
# Fit logistic regression model to estimate beta coefficients
model <- glm(exam ~ study + midterm, family = binomial, data = data)
b0 <- coef(model)[1]
b1 <- coef(model)[2]
b2 <- coef(model)[3]

# Plot data points with labels
with(data, {
  plot(study, midterm, type='n')
  text(study, midterm, exam)
})

# Plot linear decision boundary in (x1,x2)-space
tau <- 0.5
abline((log(tau / (1 - tau)) - b0) / b2, - b1 / b2, col='red', lty=3)
```

```
Call:  glm(formula = exam ~ study + midterm, family = binomial, data = data)

Coefficients:
(Intercept)        study      midterm
   -5.18982      0.57241      0.05421

Degrees of Freedom: 49 Total (i.e. Null);  47 Residual
Null Deviance:      66.41
Residual Deviance: 40.25    AIC: 46.25
```

- Logistic regression is indeed a linear model for binary classification because its decision boundary is linear.

6.7 Grouped vs Ungrouped Data

- In traditional statistics, logistic regression is often taught in the context of categorical data analysis in which all explanatory variables x_i's are categorical i.e. no continuous / quantitative x_i. Many of the statistical inference and hypothesis testing techniques (later) are valid only in this scenario.
- *Example:* Let $x \in \{1, \ldots, 5\}$ be the star rating of a hotel (viewed as a categorical variable and not a quantitative variable) and $y \in \{\text{tidy}, \text{not tidy}\}$ be a guest's review sentiment about tidiness.

```
hotel <- data.frame(star=c(1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
                           2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                           3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
                           4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
                           5, 5, 5, 5, 5, 5, 5, 5, 5, 5),
                    tidy=c(0, 0, 0, 1, 0, 0, 0, 1, 0, 0,
                           1, 0, 0, 0, 1, 0, 0, 1, 0, 1,
                           0, 1, 0, 1, 1, 0, 0, 1, 0, 1,
                           1, 0, 1, 1, 0, 1, 1, 0, 1, 1,
                           1, 1, 1, 0, 1, 1, 1, 1, 1, 1)) # 1 is tidy and 0 is not

head(hotel) # Long format i.e. one row per "review" observation unit
```

```
  star tidy
1    1    0
2    1    0
3    1    0
4    1    1
5    1    0
6    1    0
```

- R's `library(reshape2)` package provides `cast()` and `melt()` functions to convert data from long format to wide format (and vice versa).
  - **Long format**: Each row is one individual unit of observation / occurrence.
  - **Wide format**: Each row is one grouping of multiple individual observations that share a common scenario / context i.e. a particular combination of explanatory variable values.

```
library(reshape2)

(hotel.w <- dcast(hotel, star ~ tidy)) # Wide format i.e. one row per "star rating" group
```

```
  star 0 1
1    1 8 2
2    2 6 4
3    3 5 5
4    4 3 7
5    5 1 9
```

- By summing all binary responses together per each combination of categorical level of the explanatory variables, the logistic regression can now be viewed with binomial distribution instead of Bernoulli distribution i.e. the statistical model becomes

$$Y_i \sim \text{Binom}\left(n_i\,, \underbrace{\frac{1}{1+e^{-(\beta_0+\beta_1 x_{i1}+\cdots+\beta_d x_{id})}}}_{0<p_i<1}\right) \quad \text{for } i = 1, \ldots, k$$

  Here we have:
  - k is the number of groups. Each group corresponds to a distinct explanatory variable value $\vec{x}_i$.
  - n_i is the number of observations of the ith group.
  - Total number of observations is still

$$n_1 + \cdots + n_k = n$$

  - $Y_i \in \{0, \ldots, n_i\}$ is the number of success among the n_i number of observations of the ith group. It is a sum of n_i independent Bernoulli random variables. Therefore, Y_i is a binomial random variable.
- The likelihood function for binomial distribution is

$$L(\beta_0, \ldots, \beta_d) = \prod_{i=1}^{k} P(Y_i = y_i) = \prod_{i=1}^{k} \binom{n_i}{y_i} p_i^{y_i} (1-p_i)^{n_i - y_i} \quad \text{with} \quad p_i = \frac{1}{1+e^{-(\beta_0+\beta_1 x_{i1}+\cdots+\beta_d x_{id})}}$$

  The log-likelihood function is

$$l(\beta_0, \ldots, \beta_d) = \log L(\beta_0, \ldots, \beta_d) = \underbrace{\sum_{i=1}^{k} y_i \log(p_i) + (n_i - y_i)\log(1-p_i)}_{\text{numerically same as Bernoulli case}} + \underbrace{\sum_{i=1}^{k} \log\binom{n_i}{y_i}}_{\text{constant}}$$

Therefore, the maximum likelihood estimate MLE

$$\beta_0^*, \ldots, \beta_d^* = \underset{\beta_0, \ldots, \beta_d}{\arg\max}\, l(\beta_0, \ldots, \beta_d)$$

yields exactly the same estimated coefficients $\vec{\beta}^*_{\text{MLE}}$ as the previous Bernoulli formulation.

- Note this binomial formulation is based on grouping of data points that share the same explanatory variable value $\vec{x}_i$. If some explanatory variable $x_{ij} \in \mathbb{R}$ is quantitative, then it is likely that each data point has a different value for x_{ij}. In this case, each group only has $n_i = 1$ data point i.e. ungrouped.
- R's `glm(..., family=binomial)` function provides two more ways to represent the response variable in the model formula for grouped data.

```
# Method 1 (Ungrouped): One column of 1's and 0's per observation
summary(glm(tidy ~ star, family = binomial, data = hotel))
```

```
Call:
glm(formula = tidy ~ star, family = binomial, data = hotel)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 68.994  on 49  degrees of freedom
Residual deviance: 56.445  on 48  degrees of freedom
AIC: 60.445

Number of Fisher Scoring iterations: 4
```

```
# Method 2 (Grouped): Two-column matrix of counts of 1's and 0's per group
n <- hotel.w[,'1'] + hotel.w[,'0'] # Group size n_i
y <- hotel.w[,'1']
cbind(y, n - y)

summary(glm(cbind(y, n - y) ~ star, family = binomial, data = hotel.w))
```

```
     y
[1,] 2 8
[2,] 4 6
[3,] 5 5
[4,] 7 3
[5,] 9 1

Call:
glm(formula = cbind(y, n - y) ~ star, family = binomial, data = hotel.w)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12.9442  on 4  degrees of freedom
Residual deviance:  0.3946  on 3  degrees of freedom
AIC: 16.898

Number of Fisher Scoring iterations: 3
```

```
# Method 3 (Grouped): One column of proportion of 1's per group
(hotel.p <- data.frame(star=hotel.w$star,
                       n=hotel.w[,'0'] + hotel.w[,'1'],
                       y=hotel.w[,'1'] / (hotel.w[,'0'] + hotel.w[,'1'])))

summary(glm(y ~ star, family = binomial, data = hotel.p, weights = n))
```

```
  star  n   y
1    1 10 0.2
2    2 10 0.4
3    3 10 0.5
4    4 10 0.7
5    5 10 0.9

Call:
glm(formula = y ~ star, family = binomial, data = hotel.p, weights = n)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12.9442  on 4  degrees of freedom
Residual deviance:  0.3946  on 3  degrees of freedom
AIC: 16.898

Number of Fisher Scoring iterations: 3
```

- All three usages of `glm(..., family=binomial)` yield the same coefficients $\vec{\beta}^*_{\text{MLE}}$ and hence the same model equation / fitted values / predictions / residuals.
- On the other hand, the number of observations (in the sense of the number of Y_i random variables or the number of terms that appear in the likelihood function) reduces from n ungrouped individual observations to k groups. This has implication on the degrees of freedom of the model and its deviance (https://en.wikipedia.org/wiki/Deviance_(statistics)) values (later).
- In machine learning, modeling often involves at least one continuous / quantitative explanatory variable. Therefore, the data points are often not group-able.

6.8 Residual

- Generalized linear model GLM (such as logistic regression) has a few kinds of residual.
- **Response residual**: Simple difference between the observed binary outcome $y_i \in \{0, 1\}$ vs the predicted probability $\hat{p}_i$.

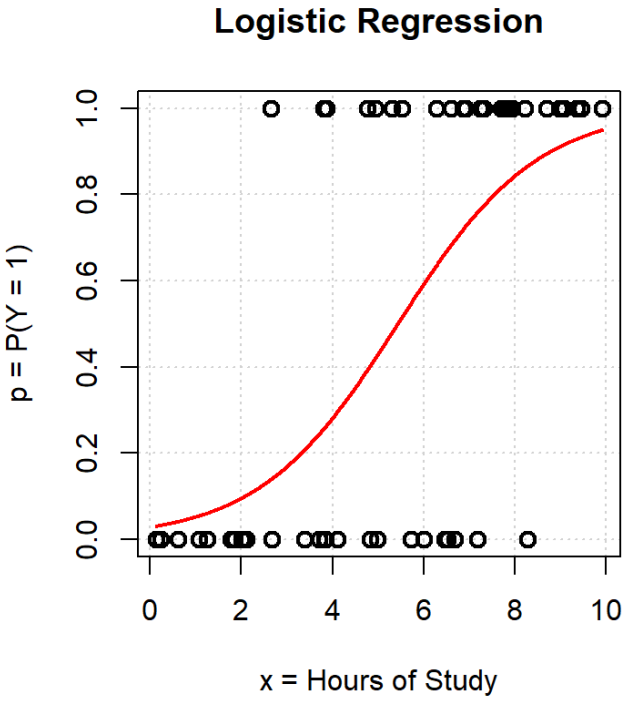
$$r_i^{\text{response}} = y_i - \hat{p}_i$$

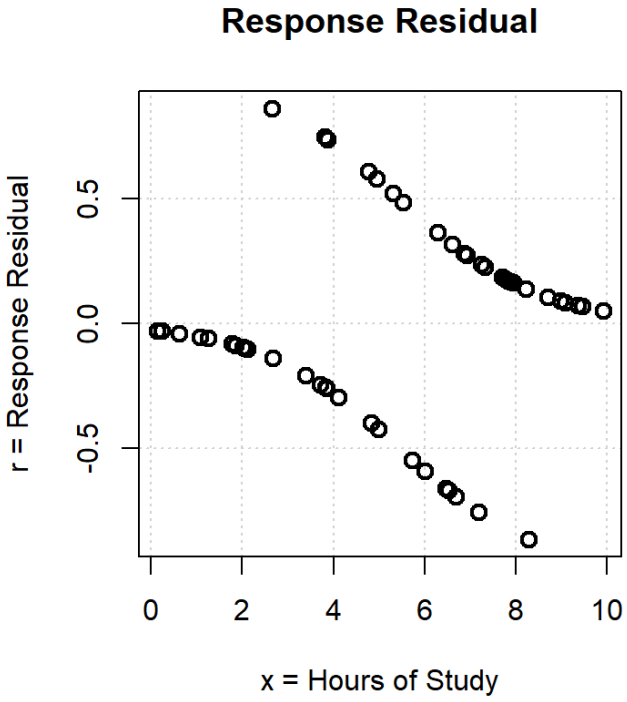
  - Easy to interpret. Same computation as linear regression.
  - However, these residuals are not on the same scale of magnitude (i.e. not standardized nor have constant variance) across the data set. For example, if $\hat{p}_i \approx 0$ or 1, then $|y_i - \hat{p}_i|$ tends to be smaller. On the other hand, $|y_i - \hat{p}_i|$ tends to be larger if $\hat{p}_i \approx 1/2$.

```
# Response residual
p <- fitted(model)

y - p
residuals(model, type='response') # See ?residuals.glm
```

```
          1           2           3           4           5           6
-0.02979261 -0.03174943 -0.04063481 -0.05418033 -0.06056313 -0.08265070
          1           2           3           4           5           6
-0.02979261 -0.03174943 -0.04063481 -0.05418033 -0.06056313 -0.08265070
```





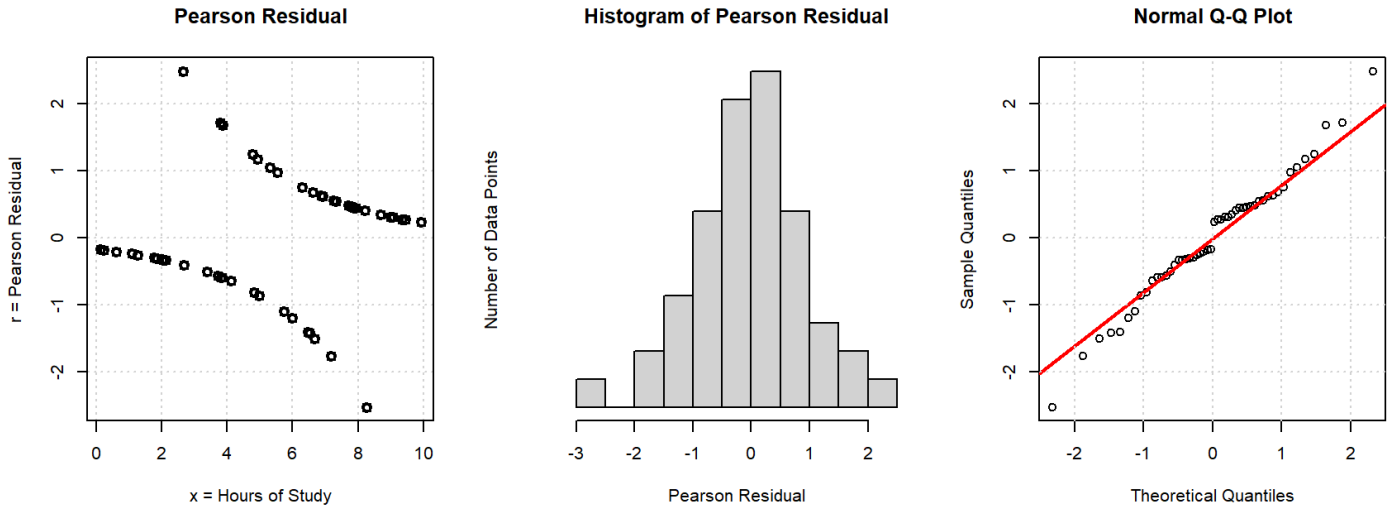
- **Pearson residual**: Attempt to standardize the response residual by dividing by its (estimated) standard error.

$$r_i^{\text{pearson}} = \frac{y_i - \hat{p}_i}{\sqrt{\hat{p}_i(1-\hat{p}_i)}} \quad \text{for ungrouped} \qquad \text{and} \qquad r_i^{\text{pearson}} = \frac{y_i - n_i\hat{p}_i}{\sqrt{n_i\hat{p}_i(1-\hat{p}_i)}} \quad \text{for grouped}$$

  - For grouped data, if each group size n_i is sufficiently large and each probability $p_i \not\approx 0$ or 1 is not too extreme, then the Pearson residuals are approximately $\mathcal{N}(0,1)$ normally distributed. (This is an example of **asymptotics** or **regularity condition** in statistics literature.) In this case, there are some proper statistical hypothesis testing techniques that use the Pearson residuals for model diagnostic.
  - For ungrouped data, the Pearson residuals are NOT necessarily approximately normally distributed (even for large number n of data points). Nonetheless, a Pearson residual $|r_i^{\text{pearson}}| \gg 2$ of large magnitude is worth investigating.

```
# Pearson residual
residuals(model, type='pearson')
(y - p) / sqrt(p * (1 - p))
```

```
         1          2          3          4          5          6
-0.1752355 -0.1810815 -0.2058056 -0.2393407 -0.2539045 -0.3001621
         1          2          3          4          5          6
-0.1752355 -0.1810815 -0.2058056 -0.2393407 -0.2539045 -0.3001621
```



- **Deviance residual**: It measures the discrepancy between the observed outcome y_i vs the predicted probability $\hat{p}_i$ using log loss

$$\text{loss}(y, p) = -\left(y \log(p) + (1-y)\log(1-p)\right) > 0$$

  - For ungrouped data, we have

$$r_i^{\text{deviance}} = s_i \sqrt{-2\left(y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right)} \qquad \text{where } s_i = \text{sign}(y_i - \hat{p}_i) = \begin{cases} +1 & \text{if } y_i = 1 \\ -1 & \text{if } y_i = 0 \end{cases}$$

    But these deviance residuals are NOT necessarily approximately normally distributed (even for large number n of data points). Nonetheless, a deviance residual $|r_i^{\text{deviance}}| \gg 2$ of large magnitude is worth investigating.
  - For grouped data, we have

$$r_i^{\text{deviance}} = s_i \sqrt{-2\left(y_i \log \hat{p}_i + (n_i - y_i)\log(1-\hat{p}_i)\right)} \qquad \text{where } s_i = \text{sign}(y_i - n_i\hat{p}_i)$$

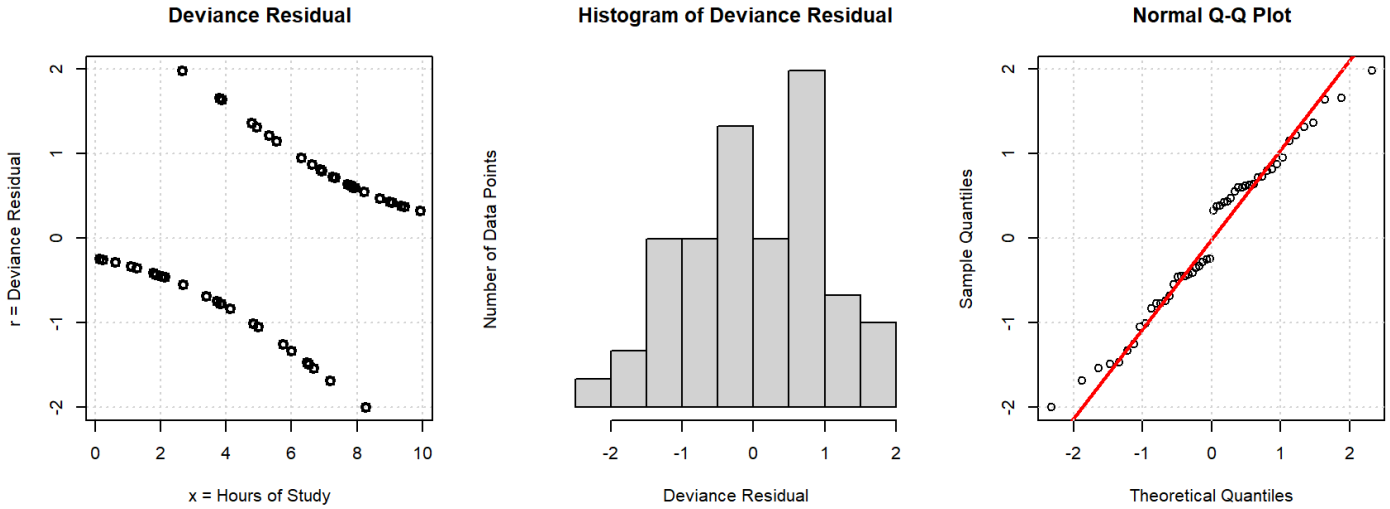
    If each group size n_i is sufficiently large and each probability $p_i \not\approx 0$ or 1 is not too extreme, then the deviance residuals are approximately $\mathcal{N}(0,1)$ normally distributed.
  - By summing the squares of deviance residuals, we obtain (a negative multiple of) the log-likelihood function i.e.

$$\sum_{i=1}^{n} (r_i^{\text{deviance}})^2 = -2 \log L(\hat{\beta}_0, \ldots, \hat{\beta}_d) = -2\, l(\hat{\beta}_0, \ldots, \hat{\beta}_d)$$

    Therefore, deviance residual r_i^{deviance} is essentially the contribution of the ith data point to the overall log-likelihood function.
  - Since the fitting of a logistic regression model is about minimizing the log-likelihood function, the deviance residuals are the actual quantities we want to make small. In this sense, deviance residual is the most "fundamental" / "instrinic" residual in the context of generalized linear model GLM. Other kinds of residual (such as response residual and Pearson residual) are merely inspirations borrowed from linear regression.

```
# Deviance residual
residuals(model) # Default  Same as resid(model)
residuals(model, type='deviance')
summary(model)$deviance.resid
```

```
        1         2         3         4         5         6
-0.2459489 -0.2540251 -0.2880398 -0.3337764 -0.3534817 -0.4153720
        1         2         3         4         5         6
-0.2459489 -0.2540251 -0.2880398 -0.3337764 -0.3534817 -0.4153720
        1         2         3         4         5         6
-0.2459489 -0.2540251 -0.2880398 -0.3337764 -0.3534817 -0.4153720
```



- **Working residual**: This is a byproduct from the internal computation at the final iteration of the Fisher scoring algorithm. (Don't worry about it.)

```
# Working residual
residuals(model, type='working')
model$residuals
```

```
        1         2         3         4         5         6
-1.030707 -1.032791 -1.042356 -1.057284 -1.064467 -1.090097
        1         2         3         4         5         6
-1.030707 -1.032791 -1.042356 -1.057284 -1.064467 -1.090097
```

- Linear regression can be viewed as a generalized linear model GLM with Gaussian distribution family i.e. `glm(..., family = gaussian)` in R. In this case, all four kinds of residual above are numerically the same. (Therefore, we did not need to introduce them separately before.)
- For logistic regression, the Pearson residual and deviance residual are different but usually similar numerically. Deviance residual has nicer statistical properties than Pearson residual.
- There are other kinds of further **standardized** / **studentized** (https://en.wikipedia.org/wiki/Studentized_residual) Pearson and deviance residual in the forms

$$r_i^{\text{std pearson}} = \frac{r_i^{\text{pearson}}}{\sqrt{1-h_i}} \qquad \text{and} \qquad r_i^{\text{std deviance}} = \frac{r_i^{\text{deviance}}}{\sqrt{1-h_i}}$$

  with some **leverage** value h_i. See R's `rstandard()` and `rstudent()` documentation. (Don't worry about these.)
- Recall that many statistcal properties about residual only hold for grouped data with large group size $n_i \gg 1$. In machine learning, data set is often ungrouped. In practice, we compute residuals and inspect their magnitudes but usually do not focus on performing statistical analysis on them.

6.9 Deviance and Goodness-of-Fit

- For linear regression, the coefficient of determination R^2 is one way to measure the goodness-of-fit of a fitted model. Does generalized linear model GLM (such as logistic regression) have such a measure? (Yes for grouped data.)
- Previously, we have

$$\underbrace{SS_{\text{total}} = \sum_{i=1}^{n}(y_i - \bar{y})^2}_{SS_{\text{null}}} \quad > \quad \underbrace{SS_{\text{error}} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2}_{SS_{\text{model}}} \quad > \quad \underbrace{0 = \sum_{i=1}^{n}(y_i - y_i)^2}_{SS_{\text{saturated}}}$$

  Here we rename SS_{total} as SS_{null} to represent the null model which always predicts by overall mean $\bar{y}$. Similarly, we rename SS_{error} as SS_{model} to represent the fitted model of interest.
- A **saturated** model is a prefect model that does not produce any error / mistake. For the ith observed data point y_i, this model would be able to output a fitted value that is exactly same as y_i. In machine learning, this phenomenon is basically **overfitting** to the extent of memorizing the training data set. In practice, this can occur if there are too many explanatory variables e.g. when the model has as many parameters as the number of data points. (It is "saturated" in this sense.)
- Coefficient of determination R^2 is the relative location (in percentage $\%$) of SS_{model} within the interval range from SS_{null} to $SS_{\text{saturated}}$.
- Now for logistic regression, we know the response residuals are not as important. Instead, let's focus on likelihood values.
- Let

$$l_{\text{null}} = \sum_{i=1}^{k} y_i \log(\bar{p}) + (n_i - y_i)\log(1-\bar{p}) + \underbrace{\sum_{i=1}^{k}\log\binom{n_i}{y_i}}_{\text{constant}}$$

  be the log-likelihood value of the null model i.e. with only the intercept term `glm(... ~ 1, family=binomial)` without any explanatory variable. It always predicts the same probability

$$p^{\text{null}} = \bar{p} = \frac{\sum_{i=1}^{k} y_i}{\sum_{i=1}^{k} n_i}$$

  based on overall proportion / mean of all data points regardless of the explanatory variable $\vec{x}$.

```
# Grouped data
(hotel.w <- dcast(hotel, star ~ tidy))
```

```
  star 0 1
1    1 8 2
2    2 6 4
3    3 5 5
4    4 3 7
5    5 1 9
```

```
# Null model
n <- rowSums(hotel.w[,c('1', '0')]) # Group size n_i
y <- hotel.w[,'1']
null.model <- glm(cbind(y, n - y) ~ 1, family = binomial, data = hotel.w)

summary(null.model)
```

```
Call:
glm(formula = cbind(y, n - y) ~ 1, family = binomial, data = hotel.w)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.1603     0.2838   0.565    0.572

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12.944  on 4  degrees of freedom
Residual deviance: 12.944  on 4  degrees of freedom
AIC: 27.448

Number of Fisher Scoring iterations: 4
```

```
# Predicted probability for null model
(p.null <- fitted(null.model))
sum(y) / sum(n) # Overall proportion
```

```
   1    2    3    4    5
0.54 0.54 0.54 0.54 0.54
[1] 0.54
```

```
# Log-likelihood of null model
logLik(null.model) # Or logLik(null.model)[1]
(l.null <- sum(y * log(p.null) + (n - y) * log(1 - p.null)) + sum(lchoose(n, y))) # See ?choose
```

```
'log Lik.' -12.72391 (df=1)
[1] -12.72391
```

- Let

$$l_{\text{model}} = \sum_{i=1}^{k} y_i \log(\hat{p}_i) + (n_i - y_i) \log(1 - \hat{p}_i) + (\text{same constant})$$

be the usual log-likelihood value of the fitted model of interest. For the ith data point y_i, this model predicts the usual fitted probability $\hat{p}_i$.

```
# Model of interest
model <- glm(cbind(y, n - y) ~ star, family = binomial, data = hotel.w)

# Log-likelihood of model
logLik(model)

p.model <- fitted(model) # Usual predicted probability p_i
(l.model <- sum(y * log(p.model) + (n - y) * log(1 - p.model)) + sum(lchoose(n, y)))
```

```
'log Lik.' -6.44911 (df=2)
[1] -6.44911
```

- Let

$$l_{\text{saturated}} = \sum_{i=1}^{k} y_i \log\left(\frac{y_i}{n_i}\right) + (n_i - y_i)\log\left(1 - \frac{y_i}{n_i}\right) + (\text{same constant})$$

be the log-likelihood value of the saturated model. For the ith group of size n_i, it has a $\text{Binom}(n_i, p_i)$ distribution and y_i observed number of success. If we could choose the perfect value for the p_i parameter based on maximum likelihood estimate MLE, then we would choose the sample proportion / mean of the group i.e.

$$p_i^{\text{saturated}} = \frac{y_i}{n_i} \quad \text{for } i = 1, \ldots, k$$

```
# Predicted probability for saturated model
(p.saturated <- y / n)

# Log-likelihood of saturated model
(l.saturated <- sum(y * log(p.saturated) + (n - y) * log(1 - p.saturated)) + sum(lchoose(n, y)))
```

```
[1] 0.2 0.4 0.5 0.7 0.9
[1] -6.25181
```

- (For ungrouped data, the log-likelihood formulae above are still valid by setting $n_i = 1$ and $k = n$ while interpreting $0 \times \log 0 = 0 \times -\infty = 0$.)
- These three log-likelihood values must satisfy the following relationship:

$$l_{\text{null}} < l_{\text{model}} < l_{\text{saturated}} \leq 0$$

Since the saturated model is perfect, its log-likelihood value $l_{\text{saturated}}$ is the upper bound of log-likelihood value over all possible models.

```
c(l.null, l.model, l.saturated)
```

```
[1] -12.72391  -6.44911  -6.25181
```

- The main idea for measuring goodness-of-fit is to use the relative location of l_{model} within the interval range from l_{null} to $l_{\text{saturated}}$. To do so, let's define **null deviance**

$$D_{\text{null}} = -2\left(l_{\text{null}} - l_{\text{saturated}}\right) > 0$$

and **residual deviance** (or simply **deviance**)

$$D_{\text{residual}} = -2\left(l_{\text{model}} - l_{\text{saturated}}\right) > 0$$

```
summary(model) # Model of interest
```

```
Call:
glm(formula = cbind(y, n - y) ~ star, family = binomial, data = hotel.w)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12.9442  on 4  degrees of freedom
Residual deviance:  0.3946  on 3  degrees of freedom
AIC: 16.898

Number of Fisher Scoring iterations: 3
```

```
# Null deviance
-2 * (l.null - l.saturated)
model$null.deviance
```

```
[1] 12.9442
[1] 12.9442
```

```
# Residual deviance
-2 * (l.model - l.saturated)
model$deviance
deviance(model)
```

```
[1] 0.3946019
[1] 0.3946019
[1] 0.3946019
```

- Some remarks on deviance (https://en.wikipedia.org/wiki/Deviance_(statistics)):
  - Multiplicative factor of -2 scales the deviance values to be more useful (later). In particular, the negative -2 makes the deviance values $D_{\text{null}} > 0$ and $D_{\text{residual}} > 0$ positive.
  - "Residual deviance" D_{residual} and "deviance residual" r_i^{deviance} (previously) are not exactly the same thing. Nonetheless, they are related

$$D_{\text{residual}} = -2\, l_{\text{model}} + \underbrace{2\, l_{\text{saturated}}}_{\text{constant}} = \underbrace{\sum_{i=1}^{k} (r_i^{\text{deviance}})^2}_{-2\, l_{\text{model}}} + \text{constant}$$

  - Since the saturated log-likelihood $l_{\text{saturated}}$ is a constant i.e. does not depend on any model parameter $\vec{\beta}$, the objective to maximize log-likelihood l_{model} is same as minimizing D_{residual}.
  - In statistics literature, there is a concept of "deviance" vs "**scaled deviance**". In logistic regression, that scaling is just 1 so they are the same thing.
- Intuitively, if the residual deviance $D_{\text{residual}} \gg 0$ is large, then the corresponding model of interest does not fit the data points adequately i.e. a lack of fit. On the other hand, a small value of residual deviance $D_{\text{residual}} \approx 0$ means the

model fits the data points as good as the perfect saturated model.

$$\overbrace{l_{\text{null}} < \underbrace{l_{\text{model}} < l_{\text{saturated}}}_{D_{\text{residual}}}}^{D_{\text{null}}} \leq 0$$

We need some benchmark to decide if D_{residual} is large or small.

- $\mathcal{X}^2$ **goodness-of-fit test**: Let's frame this as a statistical hypothesis test

$$H_0 : \text{model fits adequately} \qquad \text{vs} \qquad H_a : \text{model does not fit adequately}$$

Suppose each group size n_i is sufficiently large and each probability $p_i \not\approx 0$ or 1 is not too extreme. (This is just the previous asymptotic / regularity condition.) Under the null hypothesis H_0 i.e. if the model is adequate, then the residual deviance D_{residual} as a test statistic would have an asymptotic / approximate $\mathcal{X}^2$ reference probability distribution according to the Wilks' Theorem (https://en.wikipedia.org/wiki/Wilks%27_theorem)

$$D_{\text{residual}} \overset{d}{\approx} \mathcal{X}^2_{\text{df} = k - p} \quad \text{under } H_0$$

Here k is the number of groups and $p = 1 + d$ is the number of $\beta_0, \ldots, \beta_d$ parameters in the model. The quantity $k - p$ is called the **residual degrees of freedom**.

```
summary(model) # Model of interest
```

```
Call:
glm(formula = cbind(y, n - y) ~ star, family = binomial, data = hotel.w)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 12.9442  on 4  degrees of freedom
Residual deviance:  0.3946  on 3  degrees of freedom
AIC: 16.898

Number of Fisher Scoring iterations: 3
```

```
# Null degrees of freedom
model$df.null

k <- length(n) # Number of groups
p <- 1 # Null model only has 1 intercept parameter b0
k - p
```

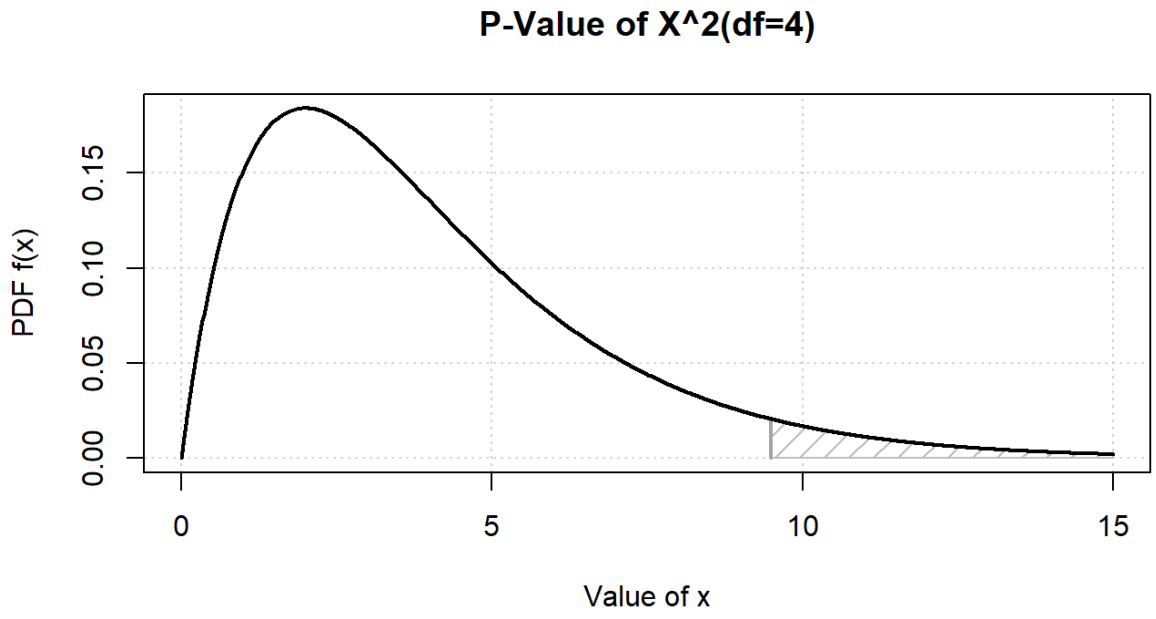
```
[1] 4
[1] 4
```

```
# Residual degrees of freedom
model$df.residual

p <- 2 # Number of parameters
k - p
```

```
[1] 3
[1] 3
```

- Now if the test statistic $D_{\text{residual}} \gg 0$ is large, then the right-tail probability of the $\mathcal{X}^2$ reference probability distribution would be small. We use this right-tail probability as the p-value to decide whether to reject H_0 or not.




```
# Compute right-tail p-value
pchisq(deviance(model), df = df.residual(model), lower.tail = FALSE) # Large p-value so cannot reject H0
```

```
[1] 0.9413553
```

- Rule-of-thumb: Recall that if a random variable $X \sim \mathcal{X}^2_{\text{df}=k}$, then $\mathbb{E}(X) = k$. Therefore, if

$$D_{\text{residual}} \gg \text{df} = k - p$$

  then the resulting p-value should be very small so we would likely reject H_0.
- For linear regression, the various concepts of sum-of-squares and residual deviance are exactly the same.

$$D_{\text{residual}} = SS_{\text{model}} = SS_{\text{error}} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

```
# Create some data
n <- 10
x <- rnorm(n)
y <- 1 + 2 * x + rnorm(n, sd=0.5)

# Fit linear regression model
model <- lm(y ~ x)

# Residual deviance
deviance(model)
sum(residuals(model) ^ 2) # SS_error = SS_model
```

```
[1] 1.221191
[1] 1.221191
```

- So far our focus is on grouped data. For ungrouped data, many of the statistical techniques above are invalid. For example:
  - Each group only has $n_i = 1$ data point. For the saturated model, the perfect predicted probability is simply the observed response y_i itself i.e.

$$p_i^{\text{saturated}} = y_i \quad \text{for } i = 1, \dots, n$$

  - Log-likelihood value of the saturated model becomes

$$l_{\text{saturated}} = \sum_{i=1}^{n} y_i \log(\underbrace{p_i^{\text{saturated}}}_{y_i}) + (1 - y_i) \log(\underbrace{1 - p_i^{\text{saturated}}}_{1-y_i}) = 0$$

    Consequently, the residual deviance D_{residual} value is different from the grouped data version.
  - Residual degrees of freedom becomes $n - p$. It is much larger than $k - p$ before.
  - Most importantly, the residual deviance D_{residual} does not have an approximate $\mathcal{X}^2$ (or any other known) reference probability distribution under the null hypothesis H_0 anymore, no matter how large the number n of data points is.

$$D_{\text{residual}} \overset{d}{\not\approx} \mathcal{X}^2_{\text{df} = \,?} \quad \text{under } H_0$$

    So residual deviance D_{residual} cannot be used to assess goodness-of-fit of the model anymore.

```
# Model of interest
model <- glm(tidy ~ star, family = binomial, data = hotel) # Ungrouped data

summary(model)
```

```
Call:
glm(formula = tidy ~ star, family = binomial, data = hotel)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.1976     0.8052  -2.729  0.00635 **
star          0.8024     0.2556   3.139  0.00170 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 68.994  on 49  degrees of freedom
Residual deviance: 56.445  on 48  degrees of freedom
AIC: 60.445

Number of Fisher Scoring iterations: 4
```

```
# Log-likelihood of model
logLik(model)

y <- hotel$tidy
p.model <- fitted(model) # Usual predicted probability p_i
(l.model <- sum(y * log(p.model) + (1 - y) * log(1 - p.model))) # Bernoulli not binomial
```

```
'log Lik.' -28.22239 (df=2)
[1] -28.22239
```

```
# Log-likelihood of saturated model
l.saturated <- 0

# Residual deviance
deviance(model)
-2 * (l.model - l.saturated)
```

```
[1] 56.44477
[1] 56.44477
```

```
# Residual degrees of freedom
model$df.residual

n <- length(y) # Total number of data points
p <- 2 # Number of parameters
n - p
```

```
[1] 48
[1] 48
```

- In practice, how can we assess goodness-of-fit of a model for ungrouped data?
    - In statistics literature, some people are inspired by the coefficient of determination R^2 of linear regression and consider the ratio

$$1 - \frac{D_{\text{residual}}}{D_{\text{null}}} = 1 - \frac{l_{\text{model}}}{l_{\text{null}}} \in [0, 1]$$

as the “percentage / proportion of deviance explained”. Note this is not the same as “percentage / proportion of variation in the response variable that is explained by the explanatory variables of the model” or the correlation between the observed response and predicted probability.

- In machine learning, people tend to focus on predictive performance and error i.e. residual. If many data points have large residuals, then this would indicate a lack of fit. Moreover, if we choose a threshold $\tau \in (0, 1)$ for the predicted probability, then we can compute a confusion matrix and related metrics e.g. accuracy / precision / recall. These are easier to interpret on an absolute scale and are likely to have direct business interest.

6.10 Model Selection with Wald Test and Likelihood Ratio Test

- Previously for linear regression, we have Wald t-test and F-test to compare a larger model vs a smaller nested model for the purpose of model selection. These two statistical hypothesis tests are equivalent. Moreover, their reference probability distributions (for computing p-value) are exact.
- Now for logistic regression, we have **Wald z-test** and $\mathcal{X}^2$ **likelihood ratio test LRT**. But their reference probability distributions are only asymptotic / approximate. It is under the assumption that either each group size n_i or the number of groups k (for grouped data) or the number of data points n (for ungrouped data) is sufficiently large.
- **Wald z-test**: To decide to select an explanatory variable x_j or not, we consider the statistical hypothesis test

$$H_0 : \beta_j = 0 \qquad \text{vs} \qquad H_a : \beta_j \neq 0$$

We compute z-statistic

$$z_j = \frac{\hat{\beta}_j - 0}{\text{s.e.}(\hat{\beta}_j)} \overset{d}{\approx} \mathcal{N}(0, 1) \quad \text{under } H_0 : \beta_j = 0$$

Some remarks:

- The $\text{s.e.}(\hat{\beta}_j)$ standard error computation is more complicated than previously for linear regression. (Don’t worry about it.)
- As the number of groups k or number of data points $n \to \infty$, the Student’s t distribution converges to the standard normal distribution i.e.

$$t_{\text{df } = n-p} \overset{d}{\approx} \mathcal{N}(0, 1)$$

Moreover, the reference probability distribution is merely asymptotic / approximate. Therefore, it is okay to use the standard normal $\mathcal{N}(0, 1)$ distribution here.

- Approximation improves as the group size / number of groups (for grouped data) / number of data points (for ungrouped data) increases. In particular, Wald z-test is valid for ungrouped data too.
- If the explanatory variable x_j is useful, then $\hat{\beta}_j \not\approx 0$ so $|z_j|$ is large. The resulting two-sided p-value would be small. So we reject H_0 accordingly.

- The `Pr(>|z|)` column in R’s `summary(model)` provides a quick overview of individual hypothesis tests $H_0 : \beta_j = 0$ for each model parameter β_j.

```
# Create some data
set.seed(1)

n <- 100
d <- 4
X <- matrix(rnorm(n * d), n, d)
colnames(X) <- paste0('x', 1:d)

beta <- c(1, 2, 4, 0, 0.5) # So x3 is not useful and x4 is somewhat
p <- 1 / (1 + exp(-(beta[1] + X %*% beta[-1]))) # P(Y=1)
y <- sapply(p, function(p) rbinom(1, 1, p))

data <- as.data.frame(cbind(y, X))

head(data)
```

```
  y         x1          x2         x3         x4
1 0 -0.6264538 -0.62036668  0.4094018  0.8936737
2 1  0.1836433  0.04211587  1.6888733 -1.0472981
3 1 -0.8356286 -0.91092165  1.5865884  1.9713374
4 1  1.5952808  0.15802877 -0.3309078 -0.3836321
5 0  0.3295078 -0.65458464 -2.2852355  1.6541453
6 1 -0.8204684  1.76728727  2.4976616  1.5122127
```

```
# Wald z-test
model <- glm(y ~ x1 + x2 + x3 + x4, family = binomial, data = data)

summary(model)
```

```

Call:
glm(formula = y ~ x1 + x2 + x3 + x4, family = binomial, data = data)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.5568     0.4315   1.290 0.196911
x1            2.3817     0.6238   3.818 0.000135 ***
x2            4.5772     1.1632   3.935 8.32e-05 ***
x3            0.3147     0.3652   0.862 0.388886
x4            0.9580     0.5423   1.766 0.077317 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 137.989  on 99  degrees of freedom
Residual deviance:  49.244  on 95  degrees of freedom
AIC: 59.244

Number of Fisher Scoring iterations: 7
```

```
# Step 1: Compute standard error for beta
C <- summary(model)$cov.scaled # Cov(estimated beta)  Same as $cov.unscaled and vcov(model)
(s.e.beta <- sqrt(diag(C))) # s.e.(beta)
```

```
(Intercept)          x1          x2          x3          x4
  0.4314875   0.6237904   1.1631953   0.3652394   0.5423289
```

```
# Step 2: Compute z-statistic
beta <- coef(model)
(z.stat <- beta / s.e.beta)
```

```
(Intercept)          x1          x2          x3          x4
  1.2904030   3.8180437   3.9350125   0.8616392   1.7664684
```

```
# Step 3: Compute (two-sided) p-value using reference N(0,1) distribution
(p.value <- 2 * pnorm(abs(z.stat), lower.tail = FALSE))
```

```
 (Intercept)           x1           x2           x3           x4
1.969108e-01 1.345141e-04 8.319235e-05 3.888861e-01 7.731730e-02
```

- Similar to linear regression, here the interpretation of a Wald test's p-value is "this explanatory variable is the last to be included into the model when all other explanatory variables are already present". Therefore, if we remove one explanatory variable from the model, then the p-values of the other remaining explanatory variables will change with respect to the reduced model. Moreover, we still need to be careful about the issue of multicollinearity and misleading inflated p-value.
- In general, to compare a larger model vs a smaller nested model, we will reformulate the previous $\mathcal{X}^2$ goodness-of-fit test with residual deviance D_{residual} in such a way that generalizes naturally to the $\mathcal{X}^2$ **likelihood ratio test LRT** for the purpose of nested model selection.
- Let's rewrite

$$D_{\text{residual}} = -2\left(l_{\text{model}} - l_{\text{saturated}}\right) = -2\left(\log L_{\text{model}} - \log L_{\text{saturated}}\right) = -2\log \underbrace{\left(\frac{L_{\text{model}}}{L_{\text{saturated}}}\right)}_{\text{likelihood ratio}} > 0$$

  For the $\mathcal{X}^2$ goodness-of-fit test, under the null hypothesis H_0 that the model of interest fits the observed data points adequately i.e. almost as good as the saturated model, this log likelihood ratio test statistic would have the reference probability distribution

$$D_{\text{residual}} = -2\log\left(\frac{L_{\text{model}}}{L_{\text{saturated}}}\right) \overset{d}{\approx} \mathcal{X}^2_{\text{df} = k - p = p^{\,\text{saturated}} - p^{\,\text{model}}} \quad \text{under } H_0$$

  This motivates us to consider statistical hypothesis testing using log likelihood ratio.
- Suppose we have a smaller model nested within a larger model. As a model includes more explanatory variables and becomes more complex, its residual deviance D_{residual} always decreases. Therefore, we have

$$D^{\text{small}}_{\text{residual}} > D^{\text{large}}_{\text{residual}}$$

  along with the numbers of model parameters

$$p^{\,\text{small}} < p^{\,\text{large}}$$

  The difference in residual deviances is

$$D^{\text{small}}_{\text{residual}} - D^{\text{large}}_{\text{residual}} = -2\left(l^{\text{small}}_{\text{model}} - l_{\text{saturated}}\right) + 2\left(l^{\text{large}}_{\text{model}} - l_{\text{saturated}}\right) = -2\log \underbrace{\left(\frac{L_{\text{small}}}{L_{\text{large}}}\right)}_{\text{likelihood ratio}} > 0$$

- $\mathcal{X}^2$ **likelihood ratio test LRT:** Let's consider the statistical hypothesis test

$$H_0 : \text{smaller model fits adequately} \qquad \text{vs} \qquad H_a : \text{smaller model fits worse}$$

  Suppose either each group size n_i or the number of groups k (for grouped data) or the number of data points n (for ungrouped data) is sufficiently large. Under the null hypothesis H_0 that the smaller model fits as good as the larger model, we have the asymptotic / approximate

$$D^{\text{small}}_{\text{residual}} - D^{\text{large}}_{\text{residual}} = -2\log\left(\frac{L_{\text{small}}}{L_{\text{large}}}\right) \overset{d}{\approx} \mathcal{X}^2_{\text{df} = p^{\text{large}} - p^{\text{small}}} \quad \text{under } H_0$$

  This reference probability distribution provides a benchmark to decide if the decrease $D^{\text{small}}_{\text{residual}} - D^{\text{large}}_{\text{residual}}$ is large enough to worth the increase in the number of explanatory variables $p^{\text{large}} - p^{\text{small}}$. This is also called **analysis of deviance**.
- This $\mathcal{X}^2$ likelihood ratio test is valid for ungrouped data too. But this does not imply nor require each individual $D^{\text{small}}_{\text{residual}}$ or $D^{\text{large}}_{\text{residual}}$ to have approximate $\mathcal{X}^2$ distribution.
- R's `anova(..., ..., test = 'Chisq')` performs the $\mathcal{X}^2$ likelihood ratio test LRT.

```
large.model <- glm(y ~ x1 + x2 + x3 + x4, family = binomial, data = data)
(D.large <- deviance(large.model))
(p.large <- 5) # Same as large.model$rank
```

```
[1] 49.24394
[1] 5
```

```
small.model <- update(large.model, ~ . - x3 - x4) # H0: b3 = b4 = 0

summary(small.model)
```

```

Call:
glm(formula = y ~ x1 + x2, family = binomial, data = data)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.5858     0.4149   1.412    0.158
x1            2.2177     0.5625   3.943 8.06e-05 ***
x2            3.8268     0.8949   4.276 1.90e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 137.989  on 99  degrees of freedom
Residual deviance:  53.327  on 97  degrees of freedom
AIC: 59.327

Number of Fisher Scoring iterations: 7
```

```
(D.small <- deviance(small.model))
(p.small <- 3) # Same as small.model$rank
```

```
[1] 53.3271
[1] 3
```

```
# Based on X^2 likelihood ratio test LRT by default instead of F-test
anova(small.model, large.model) # Same as anova(..., test = 'Chisq') and anova(..., test = 'LRT')
```

```
Analysis of Deviance Table

Model 1: y ~ x1 + x2
Model 2: y ~ x1 + x2 + x3 + x4
  Resid. Df Resid. Dev Df Deviance Pr(>Chi)
1        97     53.327
2        95     49.244  2   4.0832   0.1298
```

```
# Compute X^2-statistic and p-value manually
(diff.D <- D.small - D.large)
(diff.p <- p.large - p.small)

(p.value <- pchisq(diff.D, df = diff.p, lower.tail=FALSE)) # Not small p-value so cannot reject H0
```

```
[1] 4.083156
[1] 2
[1] 0.1298237
```

- R's `anova(..., test = 'Chisq')` can perform sequential $\mathcal{X}^2$ likelihood ratio test LRT too.
  - It starts with the (smallest) null model with only the intercept term `glm(y ~ 1, ...)`.
  - Each additional explanatory variable is included into the model in the order they appear the R's model formula `y ~ x1 + x2 + ...`.
  - The reported p-value `Pr(>Chi)` is based on the $\mathcal{X}^2$ likelihood ratio test LRT this additional explanatory variable when all previously added explanatory variables are already present in the model.

```
anova(large.model) # Same as anova(..., test = 'Chisq') and anova(..., test = 'LRT')
```

```
Analysis of Deviance Table

Model: binomial, link: logit

Response: y

Terms added sequentially (first to last)

     Df Deviance Resid. Df Resid. Dev  Pr(>Chi)
NULL                    99    137.989
x1    1   17.147        98    120.842  3.46e-05 ***
x2    1   67.515        97     53.327 < 2.2e-16 ***
x3    1    0.254        96     53.074   0.61457
x4    1    3.830        95     49.244   0.05036 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- To perform forward / backward / stepwise regression, R's `add1(..., test = 'Chisq')` and `drop1(..., test = 'Chisq')` functions would report p-value based on $\mathcal{X}^2$ likelihood ratio test LRT. However, the

stepwise regression `step()` variable selection logic is always based on Akaike Information Criterion AIC (when `k = 2`) or Bayes Information Criterion BIC (when `k = log(n)`) metric instead.

```
# One iteration of backward elimination
drop1(large.model) # Do not report p-value of X^2 likelihood ratio test LRT
```

```
Single term deletions

Model:
y ~ x1 + x2 + x3 + x4
       Df Deviance     AIC
<none>      49.244  59.244
x1      1   78.647  86.647
x2      1  120.311 128.311
x3      1   50.001  58.001
x4      1   53.074  61.074
```

```
drop1(large.model, test = 'Chisq') # Same as drop1(..., test = 'LRT')
```

```
Single term deletions

Model:
y ~ x1 + x2 + x3 + x4
       Df Deviance     AIC    LRT  Pr(>Chi)
<none>      49.244  59.244
x1      1   78.647  86.647 29.403 5.879e-08 ***
x2      1  120.311 128.311 71.068 < 2.2e-16 ***
x3      1   50.001  58.001  0.757   0.38421
x4      1   53.074  61.074  3.830   0.05036 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Backward elimination
step(large.model, direction = 'backward', k = 2, test = 'Chisq') # Use k = log(n) for BIC  Same as step
(..., test = 'LRT')
```

```
Start:  AIC=59.24
y ~ x1 + x2 + x3 + x4

       Df Deviance     AIC    LRT  Pr(>Chi)
- x3    1   50.001  58.001  0.757   0.38421
<none>      49.244  59.244
- x4    1   53.074  61.074  3.830   0.05036 .
- x1    1   78.647  86.647 29.403 5.879e-08 ***
- x2    1  120.311 128.311 71.068 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Step:  AIC=58
y ~ x1 + x2 + x4

       Df Deviance     AIC    LRT  Pr(>Chi)
<none>      50.001  58.001
- x4    1   53.327  59.327  3.326   0.06819 .
- x1    1   78.935  84.935 28.934 7.489e-08 ***
- x2    1  120.342 126.342 70.341 < 2.2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Call:  glm(formula = y ~ x1 + x2 + x4, family = binomial, data = data)

Coefficients:
(Intercept)           x1           x2           x4
     0.5340       2.3117       4.3107       0.8401

Degrees of Freedom: 99 Total (i.e. Null);  96 Residual
Null Deviance:      138
Residual Deviance: 50   AIC: 58
```

- In logistic regression, for the null hypothesis $H_0 : \beta_j = 0$ to decide to select the explanatory variable x_j or not, are the Wald z-test vs $\mathcal{X}^2$ likelihood ratio test LRT equivalent (like the Wald t-test vs F-test for linear regression)? No. They yield different p-values.

```
summary(large.model) # Wald z-test
```

```
Call:
glm(formula = y ~ x1 + x2 + x3 + x4, family = binomial, data = data)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.5568     0.4315   1.290 0.196911
x1            2.3817     0.6238   3.818 0.000135 ***
x2            4.5772     1.1632   3.935 8.32e-05 ***
x3            0.3147     0.3652   0.862 0.388886
x4            0.9580     0.5423   1.766 0.077317 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 137.989  on 99  degrees of freedom
Residual deviance:  49.244  on 95  degrees of freedom
AIC: 59.244

Number of Fisher Scoring iterations: 7
```

```
drop1(large.model, test = 'Chisq') # X^2 likelihood ratio test LRT
```

```
Single term deletions

Model:
y ~ x1 + x2 + x3 + x4
       Df Deviance     AIC    LRT  Pr(>Chi)
<none>      49.244  59.244
x1      1   78.647  86.647 29.403 5.879e-08 ***
x2      1  120.311 128.311 71.068 < 2.2e-16 ***
x3      1   50.001  58.001  0.757   0.38421
x4      1   53.074  61.074  3.830   0.05036 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

- Which one between the Wald z-test vs $\mathcal{X}^2$ likelihood ratio test LRT should we use?
  - Since the larger vs smaller models only differ by one explanatory variable x_j, the difference in their numbers of parameters is $p^{\text{large}} - p^{\text{small}} = 1$. The two hypothesis tests actually have the same asymptotic reference probability distribution (up to squaring)

$$\mathcal{N}(0,1)^2 \overset{d}{=} \mathcal{X}^2_{\text{df}=1}$$

  - Different p-values are due to different test statistic values i.e.

$$z_j = \frac{\hat{\beta}_j}{\text{s.e.}(\hat{\beta}_j)} \quad \text{vs} \quad D^{\text{small}}_{\text{residual}} - D^{\text{large}}_{\text{residual}}$$

  - In practice, $\mathcal{X}^2$ likelihood ratio test LRT is usually more accurate / reliable and hence preferred over the Wald z-test.
- For linear regression, we know the Wald t-test vs F-test are equivalent. If we view linear regression as a generalized linear model GLM, then can we also use the asymptotic / approximate $\mathcal{X}^2$ likelihood ratio test LRT for nested model selection too instead of the F-test? In theory, they are related. As the number of data points $n \to \infty$ increases, (a

scaled version of) the $\mathcal{X}^2$ distribution converges to the F-distribution.

$$\frac{1}{p^{\text{large}} - p^{\text{small}}} \mathcal{X}^2_{\text{df} = p^{\text{large}} - p^{\text{small}}} \overset{d}{\approx} F_{(p^{\text{large}} - p^{\text{small}}),\, (n - p^{\text{large}})}$$

However, in practice we should always use the exact F-test for linear regression.

- Since $\mathcal{X}^2$ likelihood ratio test LRT is only applicable to nested model selection, in order to compare two non-nested models, we should use Akaike Information Criterion AIC or Bayes Information Criterion BIC instead (similar to the approach for linear regression).

```
# Compute AIC
AIC(model)
model$aic
-2 * logLik(model) + 2 * p.large
```

```
[1] 59.24394
[1] 59.24394
'log Lik.' 59.24394 (df=5)
```

- Now we understand all the numbers reported in R's `summary()` for logistic regression model.

```
summary(model)
```

```

Call:
glm(formula = y ~ x1 + x2 + x3 + x4, family = binomial, data = data)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   0.5568     0.4315   1.290 0.196911
x1            2.3817     0.6238   3.818 0.000135 ***
x2            4.5772     1.1632   3.935 8.32e-05 ***
x3            0.3147     0.3652   0.862 0.388886
x4            0.9580     0.5423   1.766 0.077317 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 137.989  on 99  degrees of freedom
Residual deviance:  49.244  on 95  degrees of freedom
AIC: 59.244

Number of Fisher Scoring iterations: 7
```